PROGETTAZIONE DI SOFTWARE SICURO

Lezioni 3-4-5

Modelli di Sistema

Elvinia Riccobene

Università degli Studi di Milano - SSRI

Argomenti trattati

- **Modellazione di requisiti**
- **Notazioni operazioni e denotazioni**
- **FSM di Mealy**
- **FSM estese**
- **Macchine di Comunicazione**
- **Macchine di stato UML**

Il documento sui requisiti software

Si chiama Software Requirement Specification (SRS)

- SDA (Software Design Architecture) è parte di SRS

E' la dichiarazione ufficiale di ciò che è richiesto agli sviluppatori di sistemi.

Dove includere sia la definizione dei requisiti utente che la specifica dei requisiti di sistema.

NON è un documento di progettazione. Per quanto possibile, dovrebbe impostare CIÒ che il sistema dovrebbe fare piuttosto che COME dovrebbe farlo.

Qualità delle specifiche

Chiarezza/Comprensività

Non ambiguità

Consistenza

Completezza

Comprensibilità

Verificabilità/Validabilità

Tracciabilità

Modificabilità

Incrementalità

Modi di scrivere una specifica dei requisiti di sistema

Notazione	Descrizione
Frasi nel linguaggio naturale	I requisiti sono scritti utilizzando frasi numerate nel linguaggio naturale. Ogni frase deve esprimere un requisito.
Linguaggio naturale strutturato	I requisiti sono scritti nel linguaggio naturale secondo una forma o modello standard. Ogni campo in un modello fornisce le informazioni su un aspetto del requisito.
Notazioni grafiche	Modelli grafici, corredati da annotazioni di testo, sono usati per definire i requisiti funzionali del sistema. Sono comunemente utilizzati i casi d'uso e i diagrammi di sequenza del linguaggio UML (Unified Modeling Language).
Specifiche matematiche	Sono notazioni basate su concetti matematici, come gli insiemi o le macchine a stati finiti. Sebbene queste specifiche non ambigue possano ridurre l'ambiguità del documento dei requisiti, molti clienti non capiscono una specifica formale; non possono verificare che il documento dei requisiti rappresenta ciò che essi vogliono e, quindi, sono riluttanti ad accettarlo come parte del contratto.

Modellazione del sistema

E' il processo di sviluppo di modelli astratti di un sistema.

Può consistere in più di un modello: ogni modello presenta una visione o una prospettiva diversa di quel sistema.

La modellazione del sistema aiuta l'analista a comprendere la funzionalità del sistema e i modelli vengono utilizzati per comunicare con i clienti.

Modelli di sistema esistenti e pianificati

I modelli vengono utilizzati durante l'ingegneria dei requisiti per sistemi sia esistenti che nuovi:

- Esistente:
  - aiutano a chiarire ciò che il sistema esistente fa e possono essere utilizzati come base per discutere i suoi punti di forza e di debolezza. Portano ai requisiti per il nuovo sistema.
- Nuovo:
  - aiutano a spiegare i requisiti proposti ad altre parti interessate del sistema; sono usati per discutere le proposte di progettazione; sono usati a scopi di documentazione.

In un processo di *progettazione model-driven*, è possibile generare un'implementazione di sistema completa o parziale dal modello di sistema.

Linguaggi per la specifica

Informali

- Linguaggio naturale con +/- strutture e tabelle

Semi-formali

- notazioni grafici (es. UML)

Formali

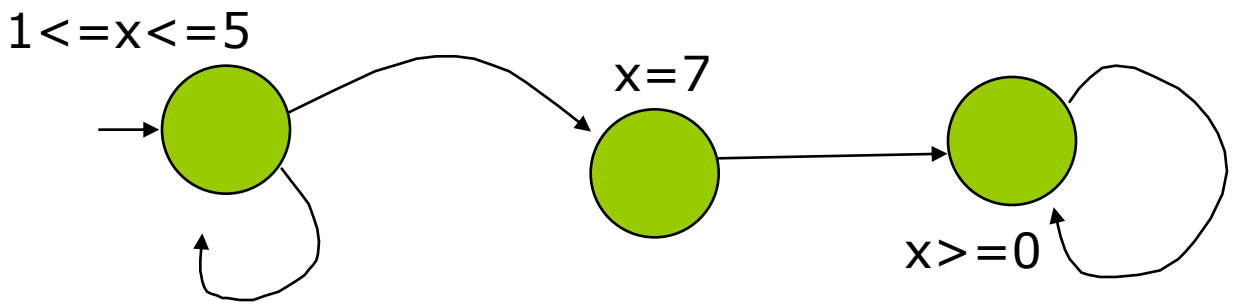
  - Formalismo è una notazione rigorosa/matematica
- Formalismi operazionali
- Formalismi dichiarativi

Diversi modi di specifica

- **Informale:**

  Il valore di x sarà tra 1 e 5, finché ad un certo momento diventa 7. In ogni caso non sarà mai negativo.

- **Grafico:**



- **Formale:** (logica temporale)

  $(1<=x<=5 \ U \ x=7) \wedge [] \ x>=0$

Diversi formalismi (1)

Formalismi operazionali:

- definiscono il sistema descrivendone il comportamento (normalmente mediante un modello) come se eseguito da una macchina astratta

Formalismi dichiarativi:

- definiscono il sistema dichiarando le proprietà che esso deve avere

Diversi formalismi (2)

<u>Esempio</u>: definizione di ellisse

- Definizione operazionale

  E' l'insieme dei punti del piano che si ottiene muovendosi in modo che la somma delle distanze tra il punto e due punti fissi p1 e p2 rimanga invariata

- Definizione dichiarativa

  E' l'insieme dei punti del piano che soddisfano l'equazione $ax^2 + by^2 + c = 0$

Diversi formalismi (3)

<u>Esempio</u>: definizione di array ordinato

- Definizione operazionale

  Sia *a* un array di *n* elementi. Il risultato del suo ordinamento è un array *b* di *n* elementi tali che il primo elemento di *b* è il minimo di *a* (se diversi elementi hanno lo stesso valore, uno di essi è selezionato); il secondo elemento di *b* è il minimo dell' array di *n-1* elementi ottenuto da *a* rimuovendo il suo elemento minimale; e così via fino a che gli *n* elementi di *a* sono stati rimossi

- Definizione dichiarativa

  Il risultato dell'ordinamento di un array *a* è un array *b* che è una permutazione di *a* ed è ordinato

Formalismi operazionali

L' approccio operazionale fornisce una rappresentazione più intuitiva poiché più simile al modo di ragionare della mente umana

- Facilita la prototipazione
- Facilita la validazione

Formalismi operazioni che vedremo:

- FSM di Mealy
- FSM estese
- Macchine di Comunicazione
- Macchine di stato UML

Formalismi dichiarativi

L' approccio dichiarativo fornisce una rappresentazione che non si presta ad ambiguità, ma e' più difficile da comprendere e sviluppare

- Facilita la verifica

Formalismi dichiarativi che vedremo:

- Design-by-Contract e JML

MACCHINE A STATI FINITI

Macchina a Stati Finiti (1)

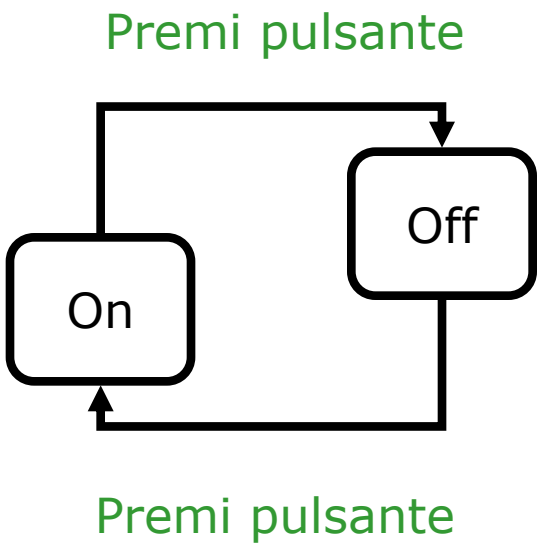
Una macchina a stati finiti, abbreviata FSM (Finite State Machine), è una notazione formale che permette la rappresentazione astratta del comportamento di un sistema

Le FSM hanno:

- una rigorosa definizione matematica
- una intuitiva rappresentazione grafica tramite diagrammi di stato

FSM: un primo esempio

<u>Esempio</u>: comportamento di una lampadina



- I nodi rappresentano gli stati del sistema
- Gli archi rappresentano il passaggio di stato

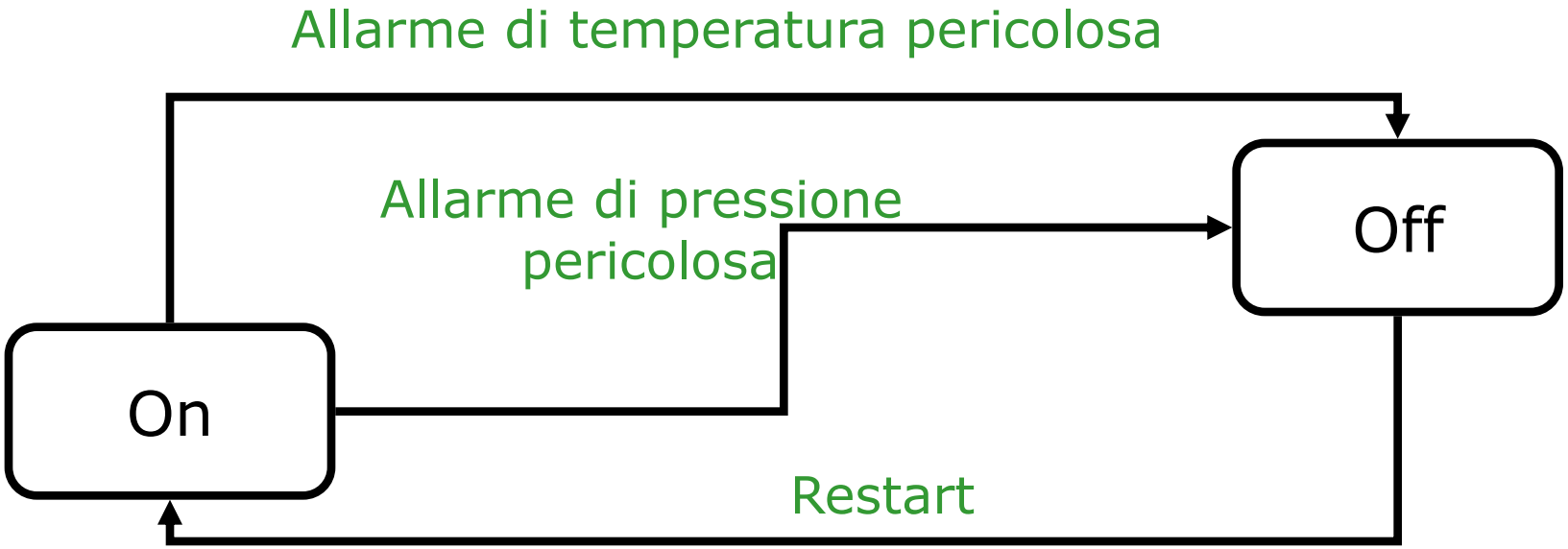
Macchina a Stati Finiti (2)

Dall'esempio si evince che le FSM modellano le configurazioni istantanee di un sistema tramite stati (*nodi* del diagramma) e le operazioni del sistema tramite transizioni (*archi* del diagramma)

Le operazioni possono ricevere input (FSM di base) ed eventualmente produrre output (FSM di Mealy)

Esempio di una FSM

Sistema di controllo di impianto chimico



Modellare con FSM

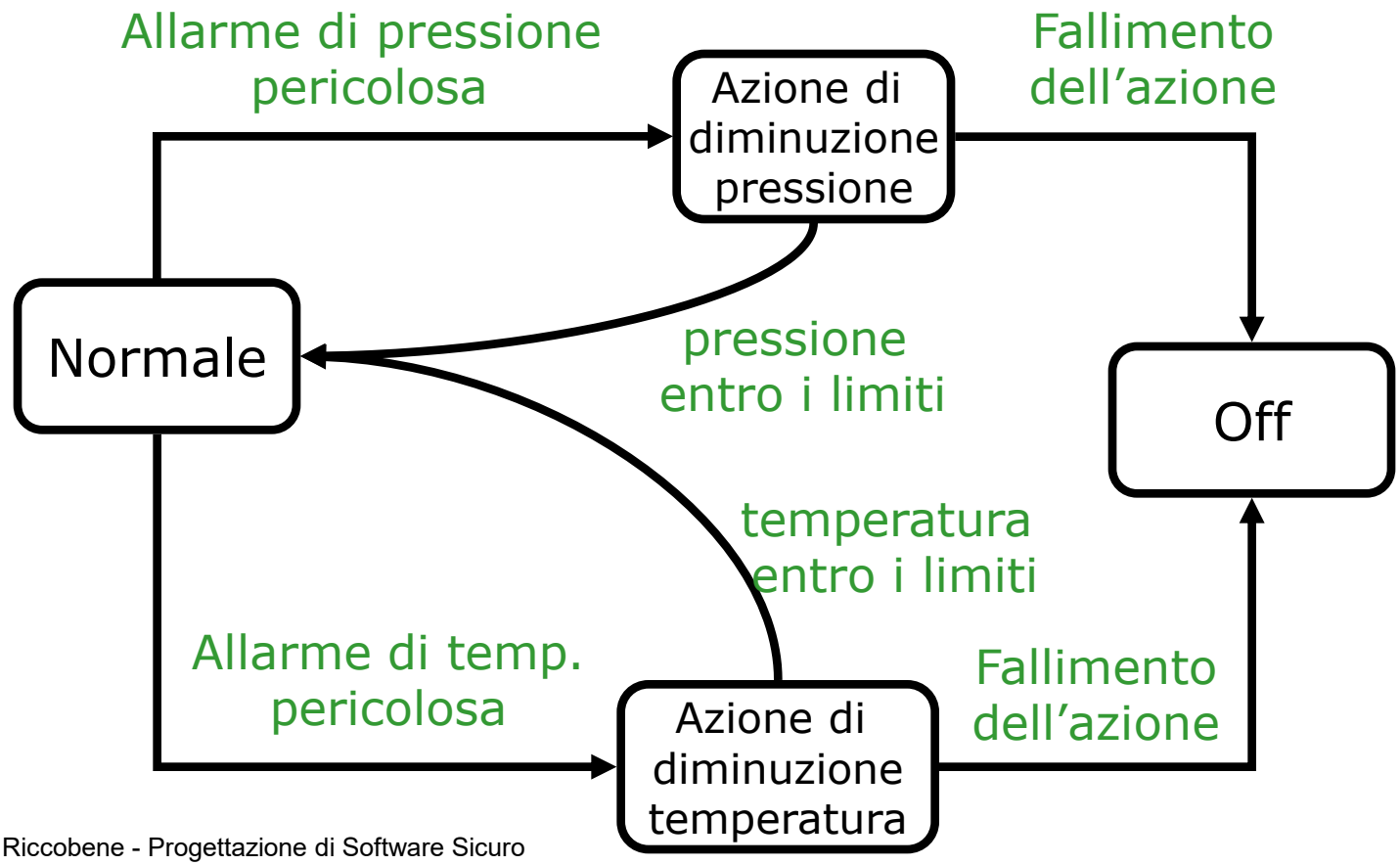
Il modello FSM di un sistema non è unico

Modelli diversi in base al livello di astrazione/dettaglio considerato

- Confronta il modello FSM del sistema di controllo di un impianto chimico con il modello successivo che considera un diverso livello di astrazione

Esempio di una FSM

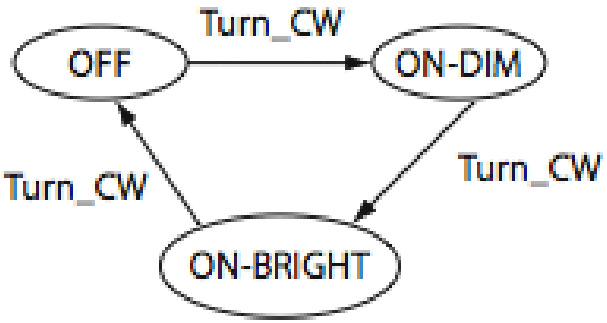
Sistema di controllo di impianto chimico: un livello di astrazione più basso



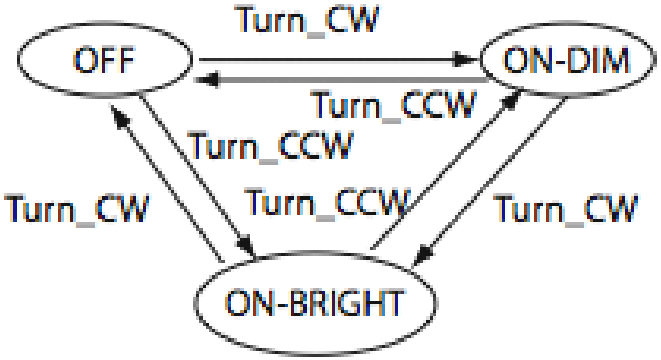
Esempio di una FSM

Lampada a tre stati:

(a) l'interruttore può essere girato in senso orario
(b) l'interruttore può essere girato in senso orario ed antiorario



(a)



(b)

Finite State Machine: definizione

Una FSM è una tupla (S, I, δ)

- S : insieme finito di stati
- I : insieme finito di eventi di input
- δ : S x I ->S : funzione di transizione

FSM: rappresentazione grafica

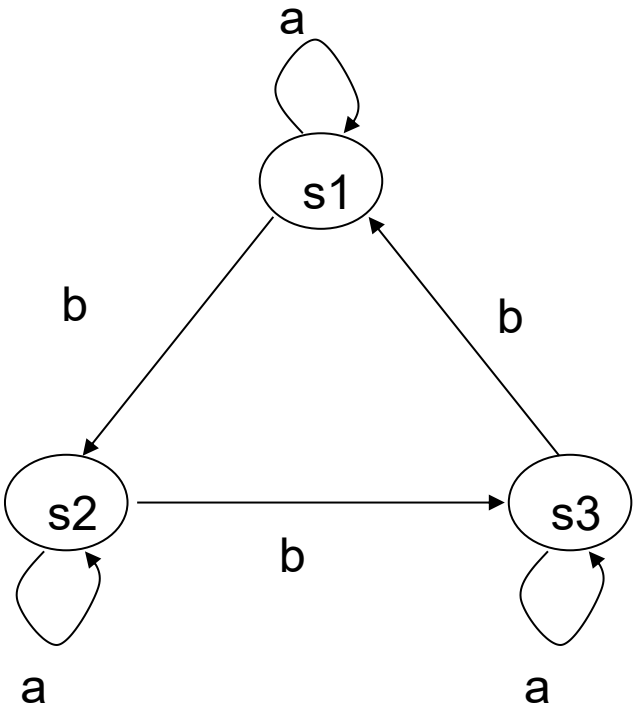
Un diagramma di stato è un grafo direzionato i cui nodi rappresentano gli stati ed i cui archi, etichettati dagli input, rappresentano le transizioni di stato

Esempio:

S = {s1, s2, s3}

I = {a, b}

δ= { (s1,a,s1), (s1,b,s2),
(s2,a,s2), (s2,b,s3),
(s3,a,s3), (s3,b,s1)}



Esempio

Sistema di controllo di impianto chimico (caso raffinato)

S = {Normale, Off, Azione di diminuzione pressione, Azione di diminuzione temperatura}

I = {Allarme di pressione pericolosa, Allarme di temp. pericolosa, Fallimento dell'azione_P, Fallimento dell'azione_T, pressione entro i limiti, temperatura entro i limiti}

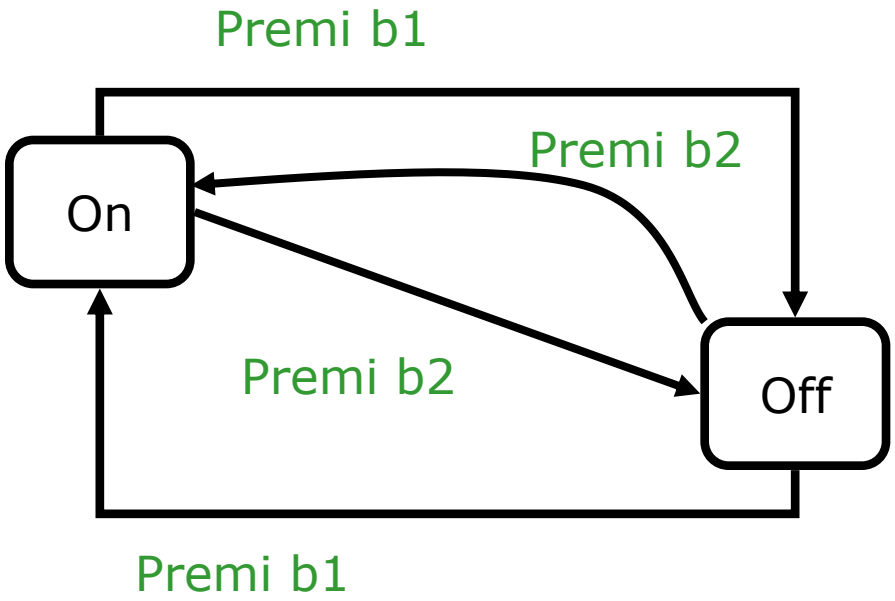
δ = {(Normale, Allarme di pressione pericolosa, Azione di diminuzione pressione), (Azione di diminuzione pressione, Fallimento dell'azione_P, Off), (Azione di diminuzione pressione, pressione entro i limiti, Normale), ecc.}

Esercizio 1

Usando una FSM descrivere un sistema di una lampada e due bottoni avente il seguente comportamento: se la lampada è spenta, il pigiare uno dei due tasti causa l'accensione della lampada.

Soluzione esercizio 1

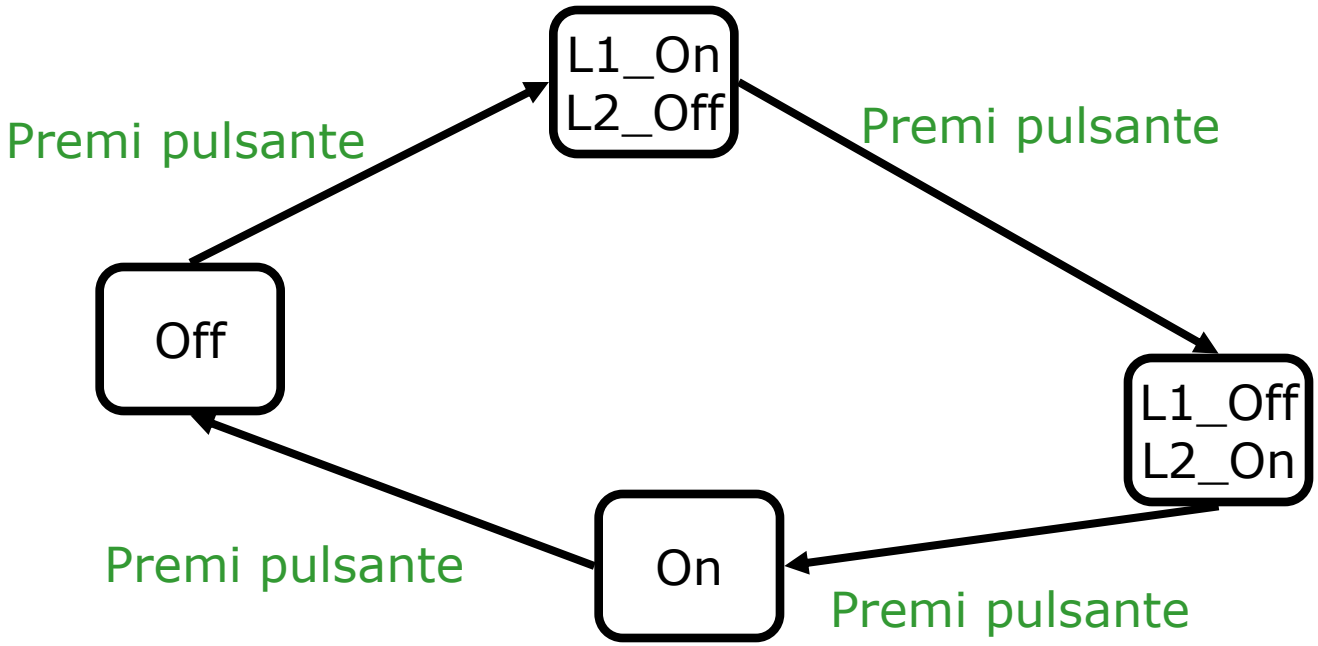
Tasti: b1 e b2



Esercizio 2

Descrivere un sistema con 2 lampade ed un bottone. Quando la luce è off, pigiare il bottone causa l' accensione della prima lampadina. Pigiare nuovamente il bottone causa l' accensione della seconda lampadina e lo spegnimento della prima. Pigiare nuovamente il bottone causa l' accensione di entrambe le lampadine, pigiare nuovamente causa lo spegnimento di entrambe le lampadine.

Soluzione esercizio 2



On= L1_On ed L2_On

Macchine a stati finiti con output

Una FSM (S, I, δ) con output è:

- una macchina di Mealy se è una FSM che produce un output per **ciascuna transizione**

Macchina di Mealy

Una macchina di Mealy è una tupla

$(S, I, O, \delta, \lambda)$

- S : insieme finito di stati
- I : insieme finito di eventi di input
- O : insieme finito di eventi di output
- δ : S x I ->S : funzione di transizione
- λ : S x I ->O : funzione di output

Spesso è anche fissato lo stato iniziale $s_0 \in S$

FSM di Mealy: rappresentazione grafica

Nei diagrammi che rappresentano una FSM di Mealy, ogni arco è etichettato da i/o

- **i** denota un simbolo di input ed è anche noto come evento di input
- **o** denota un simbolo di output ed è anche noto come evento di output
  - Un evento di output può anche essere una azione della macchina

FSM di Mealy: rappresentazione grafica

Esempio:

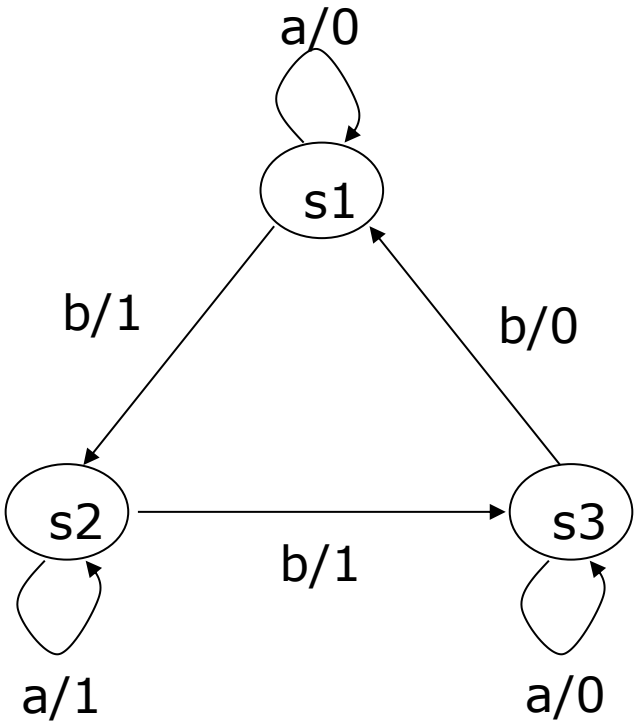
$S = \{s1, s2, s3\}$

$I = \{a, b\}$

$O = \{0, 1\}$

$\delta = \{ (s1,a,s1), (s1,b,s2),$
$(s2,a,s2), (s2,b,s3),$
$(s3,a,s3), (s3,b,s1)\}$

$\lambda = \{ (s1,a,0), (s1,b,1),$
$(s2,a,1), (s2,b,1),$
$(s3,a,0), (s3,b,0)\}$



FSM di Mealy

È utile vedere una FSM di Mealy come la tupla (S, I, O, T)

- S : insieme finito di stati
- I : insieme finito di eventi di input
- O : insieme finito di eventi di output
- T : insieme finito di transizioni
  - Una transizione è una tupla (s, i, o, s')
    - s : stato sorgente
    - i : evento di input
    - o : evento di output
    - s' : stato target

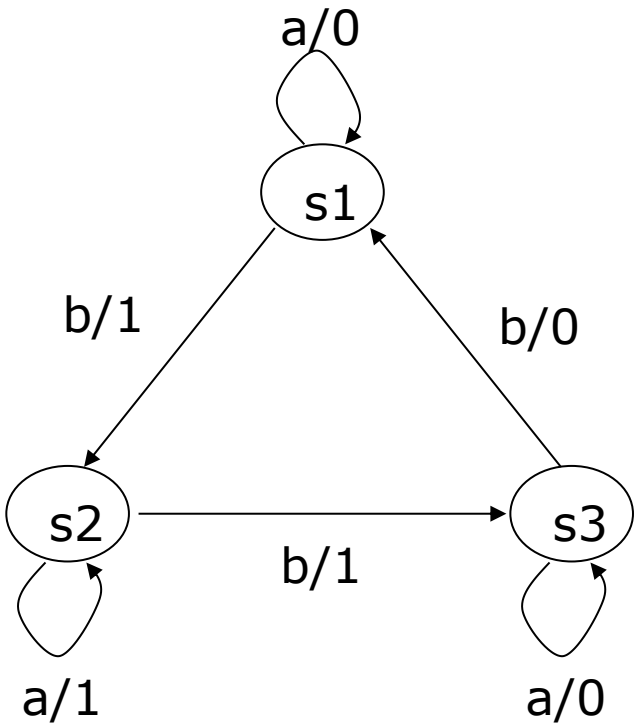
FSM di Mealy: esempio

S = {s1, s2, s3}

I = {a, b}

O = {0, 1}

T = {(s1,a,0,s1),
(s1,b,1,s2),
(s2,a,1,s2),
(s2,b,1,s3),
(s3,a,0,s3),
(s3,b,0,s1)}



Nota importante

Nel seguito per FSM intenderemo macchine di Mealy

Per esse utilizzeremo la definizione come la tupla (S, I, O, T)

Esercizio

Modellare con una FSM il comportamento di una sbarra che consente l'accesso/l'uscita di un parcheggio.

Per accedere al parcheggio, il semaforo deve essere verde. Mentre la sbarra è aperta, il semaforo è rosso.

Quando l'auto è entrata, la sbarra si chiude ed il semaforo ritorna verde.

Per uscire dal parcheggio, l'autista deve inserire il biglietto pre-pagato nel dispositivo che comanda la sbarra.

Quando l'auto è uscita la sbarra si chiude.

Soluzione esercizio

m_{en} : segnale di presenza dell'auto **dentro** il parcheggio

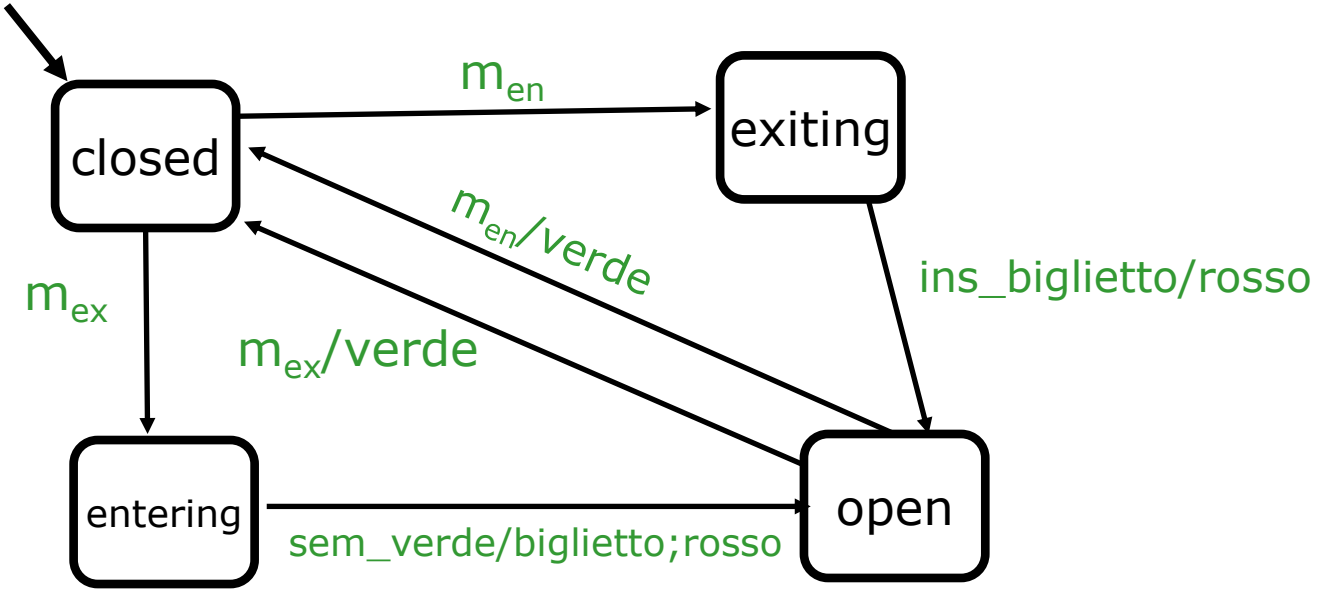
m_{ex}: segnale di presenza dell'auto **fuori** il parcheggio

sem_verde: segnale di accesso dal semaforo

ins_biglietto : segnale di inserimento biglietto

biglietto: azione di emissione del biglietto

verde, rosso: azioni x colore semaforo



PER ESERCITARSI

(POSSIBILI SOLUZIONI A SARA.ZAMBONI@STUDENTI.UNIMI.IT)

Esercizio : FSM con input

Modellare con una FSM il comportamento di una porta con la chiave inserita. I possibili eventi sono:

1. **Aprire la porta (a)**
2. **Chiudere la porta (c)**
3. **Girare la chiave per aprire (ga)**
4. **Girare la chiave per chiudere (gc)**

Esercizio: FSM con input

Modellare con una FSM il comportamento di una porta girevole in modalità singolo bottone.

- **Se la porta è chiusa, premendo il bottone la porta inizia ad aprirsi.**
- **Se si preme ancora una volta il bottone prima che la porta si apre del tutto, la porta inizia a chiudersi.**
- **Se, invece, permettiamo alla porta di aprirsi del tutto, una volta aperta la porta inizia a chiudersi automaticamente dopo un timeout di 2-secondi.**
- **Per impedire la chiusura automatica e mantenere la porta aperta occorre premere il bottone; a questo punto, un'ulteriore pressione del bottone, fa si che la porta inizi a chiudersi.**
- **Se si preme ancora una volta il bottone prima che la porta si chiuda del tutto, la porta inizia ad aprirsi.**

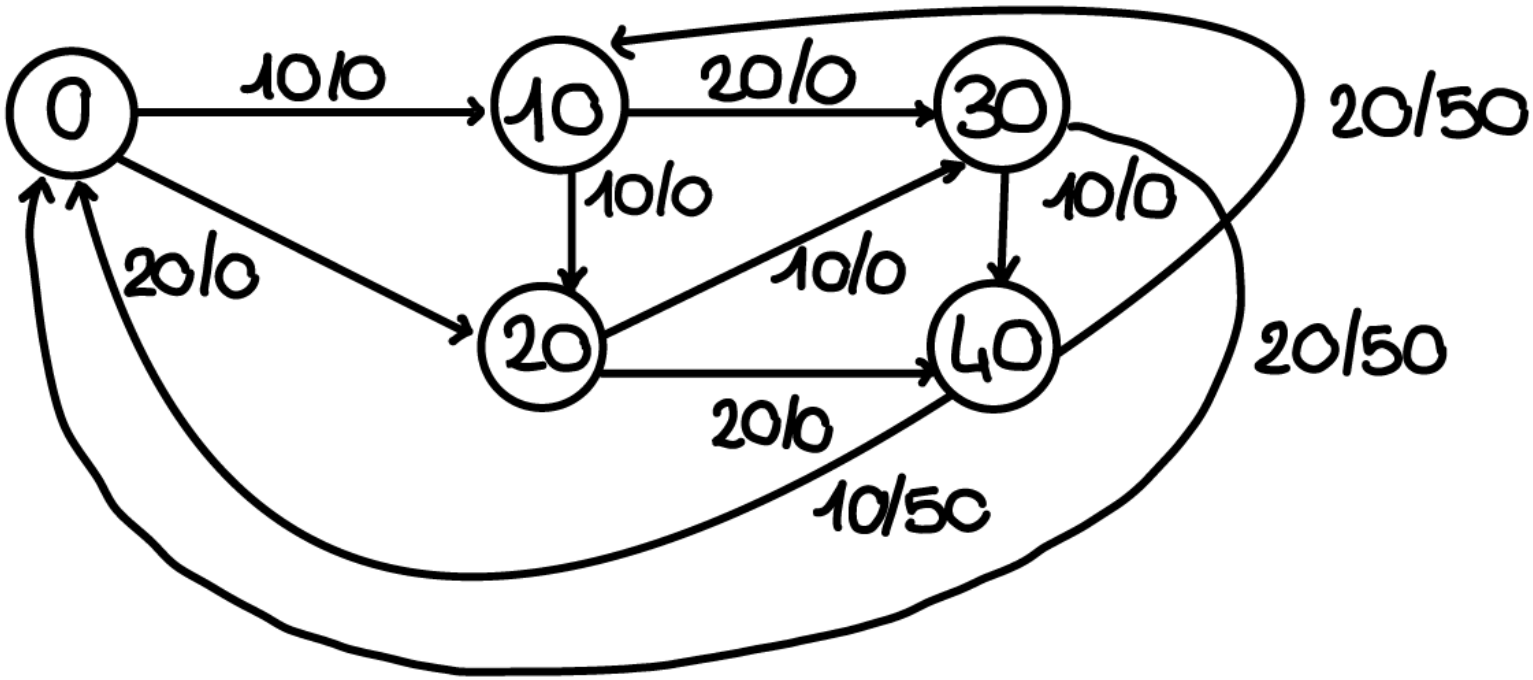
Esercizio : FSM con input

Modellare con una FSM a stati finiti il comportamento di un CD-player. Il CD-Player ha 3 tasti: OPEN/CLOSE, PLAY e STOP. L'utente può mettere o rimuovere un CD sul piatto. Quando l'utente preme PLAY e c'e' un CD sul piatto, il CD-player suona. Quando l'utente preme OPEN/CLOSE, il CD-player si apre o chiude. Quando l'utente preme STOP, il CD-player smette di suonare.

Esercizio : FSM con output

Modellare con una FSM un automa che fornisce monete da 50 centesimi in cambio di monete da 10 e 20 centesimi. La macchina non fornisce resto.

Soluzione del cambia-monete



Esercizio : FSM con output

Modella con una FSM il comportamento di un orologio digitale.

Un interruttore ON/OFF permette di accendere e spegnere l'orologio.

Il tasto set permette di aggiustare l'ora mediante i tasti m ed h per minuti ed ore.

Durante la fase di aggiustamento il display segnala l'ora in modo lampeggiante.

Settato l'orario, tramite set, l'orologio passa in modalità normale.

RIPRENDIAMO….

Modellare con FSMs

Sono utilizzate per modellare

- GUIs
- Protocolli di rete
- Pacemakers e teller machines
- Applicazioni WEB
- Software di sicurezza
- Sistemi embedded

Modellare con FSMs: limiti

Non tutti i requisiti possono essere specificati con una FSM

Tra i requisiti non specificabili in FSM:

- requisiti real time
- requisiti riguardanti la memoria della macchina
- requisiti riguardanti sotto-computazioni

MACCHINE A STATI FINITI ESTESE

Macchine a stati finiti estese (1)

- Le FSM modellano le operazioni di un sistema tramite il concetto di transizione
- Le operazioni ricevono input e possono produrre un output
- È spesso indispensabile poter modellare la memoria interna di una macchina
- Le EFSM (*Extended Finite State Machines*) estendono le FSM con il concetto di variabile

Macchine a Stati Finiti estese (2)

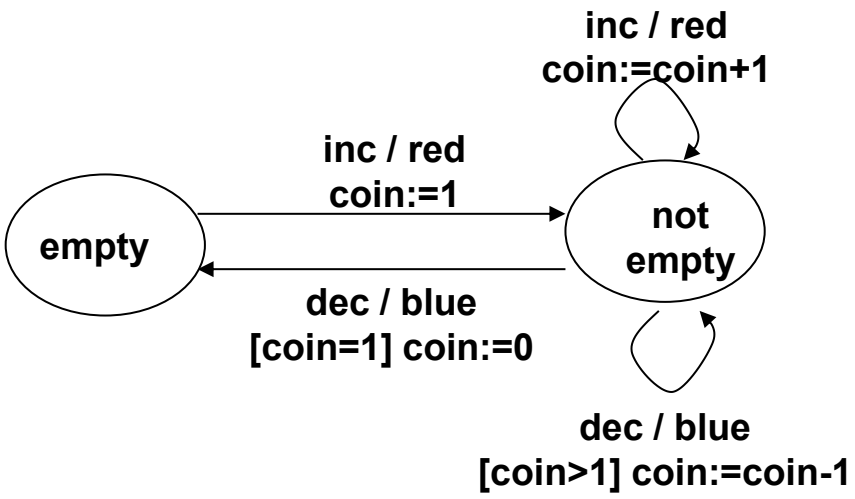
Una EFSM è una tupla (S, I, O, V, T)

- S : insieme finito di stati
- I : insieme finito di eventi di input
- O : insieme finito di eventi di output
- V : insieme finito di variabili
- T : insieme finito di transizioni
  - Una transizione è una tupla (s, i, o, g, a, s')
    - s : stato sorgente
    - i : evento di input
    - o : evento di output
    - g : predicato sulle variabili in V, detto **guardia**
    - a : assegnamento ad una variabile in V, detto **azione**
    - s' : stato target

Esempio di FSM estesa (1)

Salvadanaio elettronico :

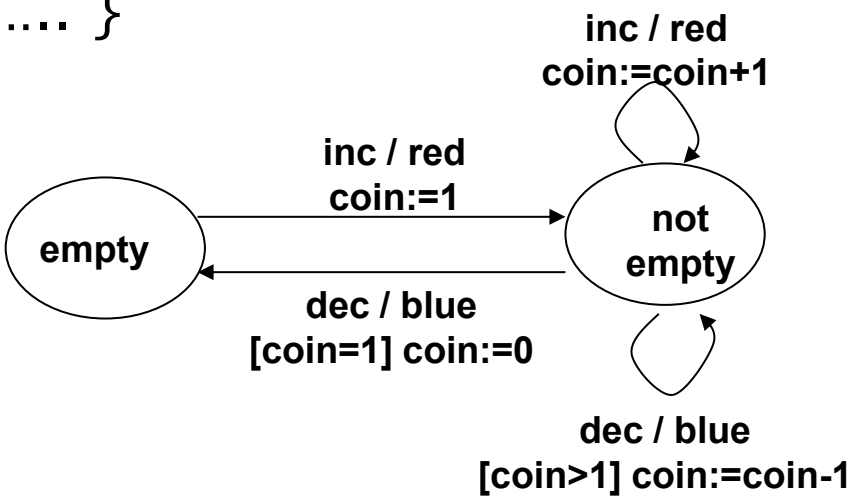
- inserendo monete (inc) il valore di coin viene incrementato
- emettendo monete (dec) il valore di coin viene decrementato
- la luce diventa red quando si inseriscono monete, diventa blue quando si richiedono monete
- la macchina non restituisce monete quando è vuota



Esempio di FSM estesa (2)

Salvadanaio elettronico: formalmente

- S = {empty, not-empty}
- I = {inc, dec}
- O ={red, blue}
- V ={coin}
- T ={(empty, inc, red, coin:=1, not-empty),
  (not-empty, dec, blue, coin=1, coin:=0, empty),
  ….. }



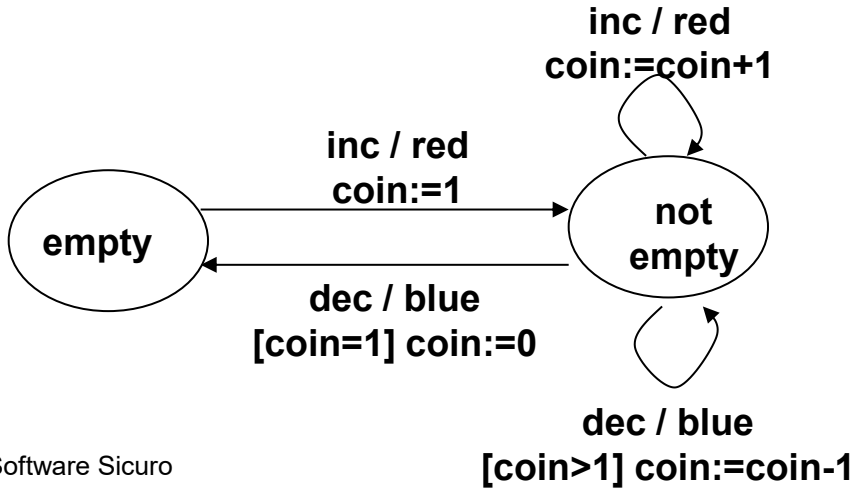
Stato globale

Per una EFSM (S, I, O, V, T), una coppia (s, σ) è detta stato globale se

- s è uno stato e
- σ è una valutazione su V

Esempio:

- (empty, coin=0), (notempty, coin=1)



Transizione globale

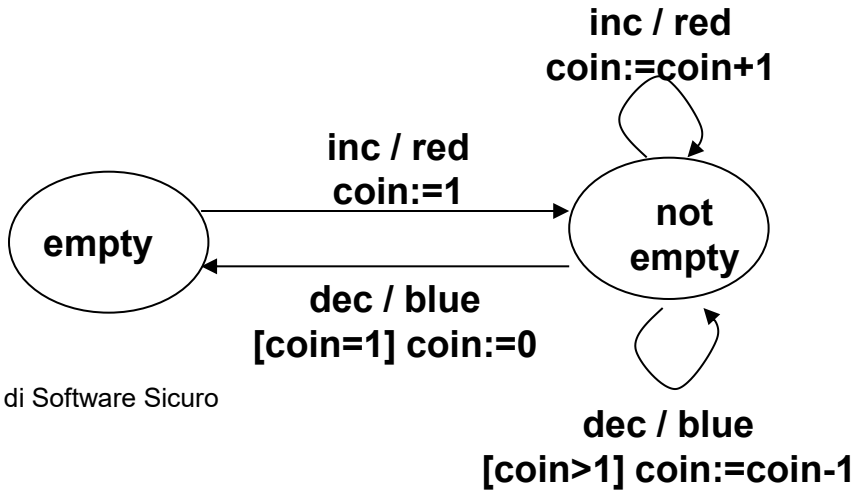
Per una EFSM (S, I, O, V, T), una tupla

((s, σ), i, o, (s', σ')) è detta transizione globale se

- esiste una transizione (s, i, o, g, a, s') tale che
  - σ soddisfa la guardia g e
  - σ'(v) = σ(exp) dove l'azione a è v:=exp

Esempio:

- ((empty, coin=0), inc, red, (notempty, coin=1))



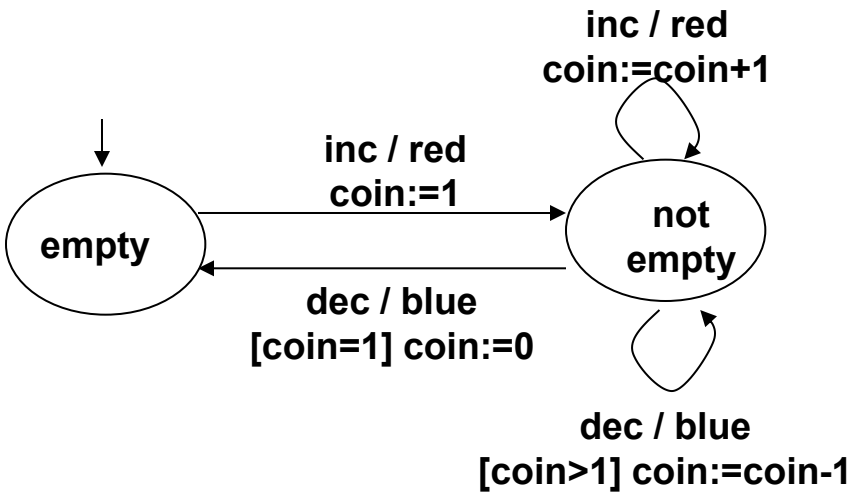
Grafo di raggiungibilità

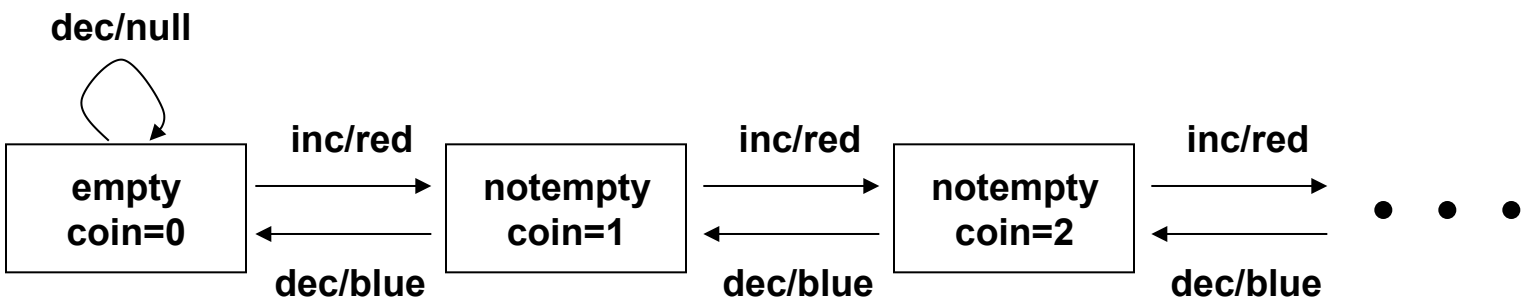
Il grafo di raggiungibilità di una EFSM (S, I, O, V, T) è un grafo direzionato in cui:

- i nodi sono gli stati globali
- gli archi sono le transizioni globali

Se le variabili hanno un range finito, il grafo di raggiungibilità di una EFSM è una FSM

Esempio


inc / red
coin:=coin+1
inc / red
coin:=1
empty
not
empty
dec / blue
[coin=1] coin:=0
dec / blue
[coin>1] coin:=coin-1


dec/null
empty
coin=0
inc/red
dec/blue
notempty
coin=1
inc/red
dec/blue
notempty
coin=2
inc/red
dec/blue
• • •

Esercizio

Modellare con una EFSM il comportamento di un dispositivo che funziona come pompa di calore e condizionatore.

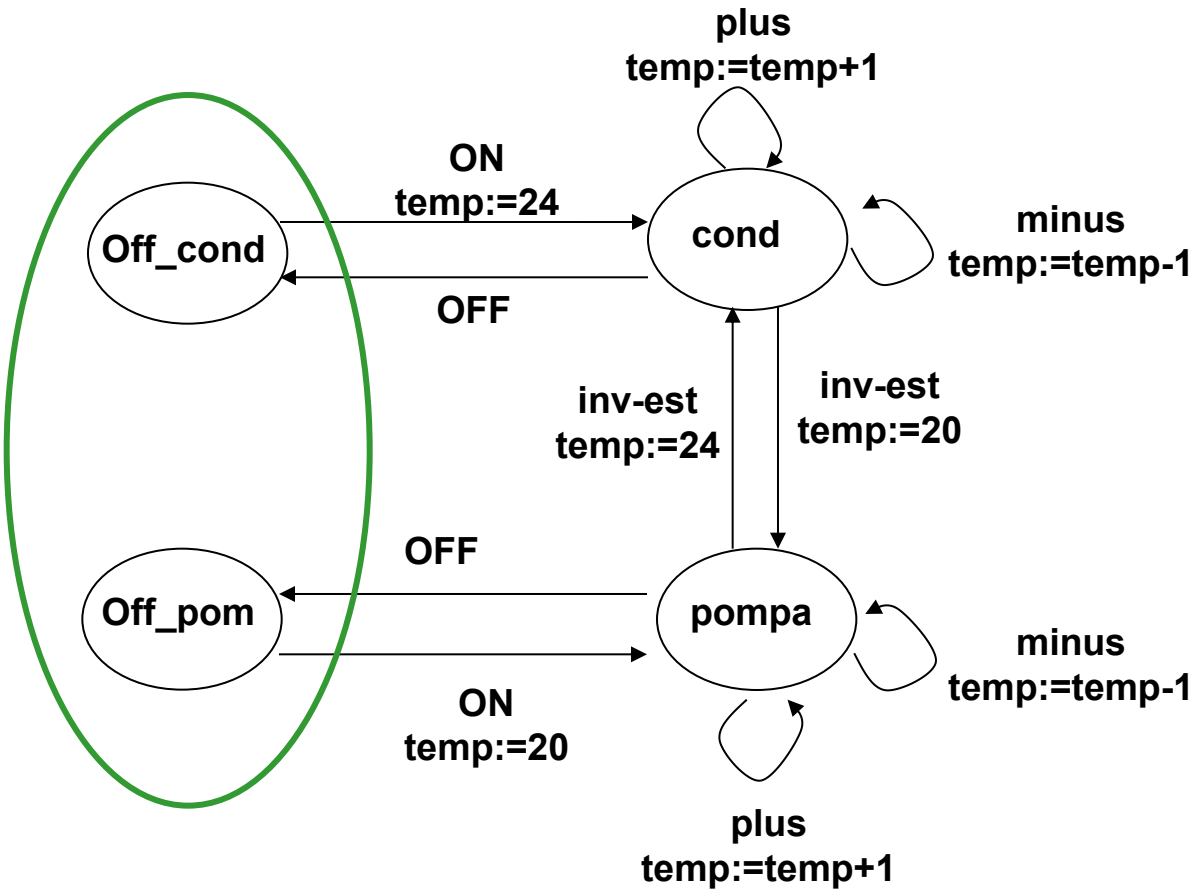
Il dispositivo è inizialmente spento e viene settato come pompa di calore o come condizionatore dal tasto inverno/estate.

Se spento, il dispositivo memorizza l'ultima modalità.

Se il dispositivo funziona come pompa di calore, la temperatura viene settata a 20 gradi e può essere aumentata o diminuita mediante telecomando.

Se il dispositivo funziona come condizionatore, la temperatura viene settata a 24 gradi e può essere aumentata o diminuita mediante telecomando.

Soluzione

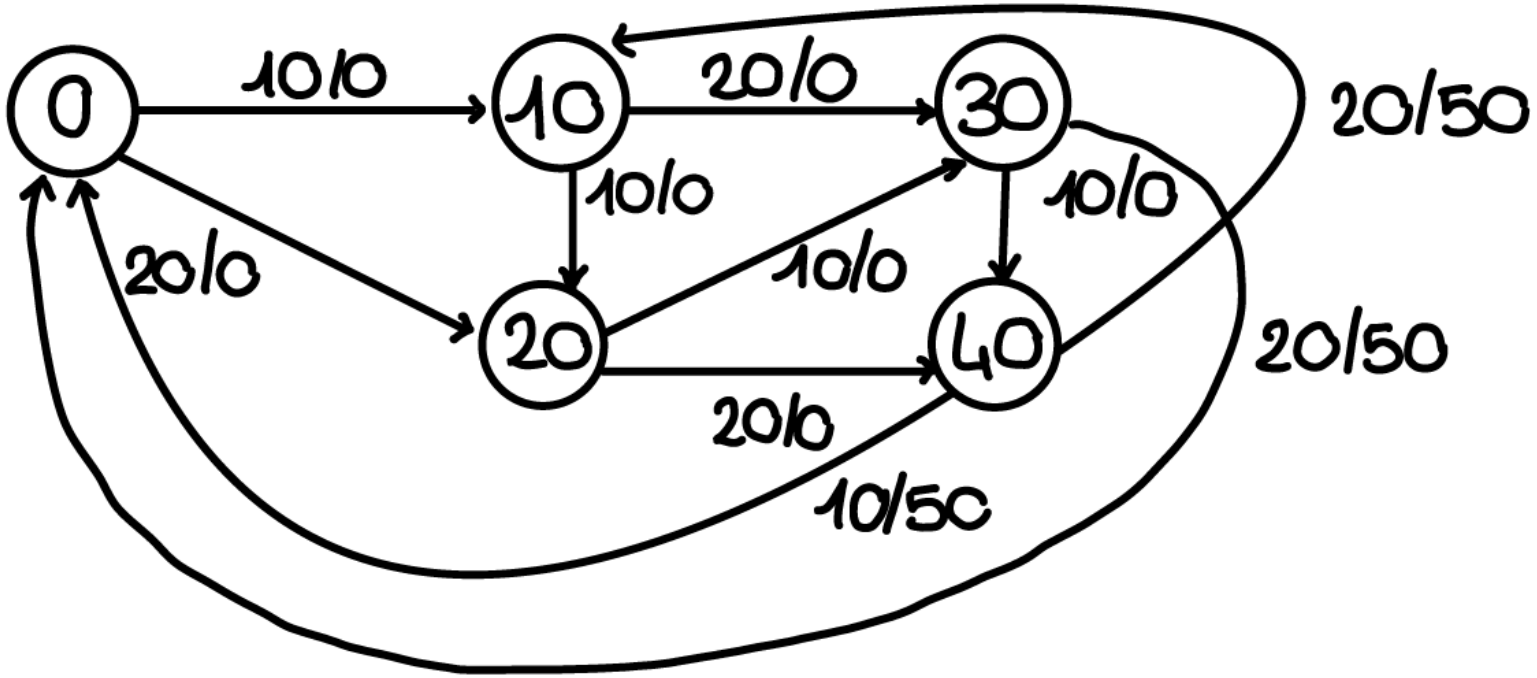

plus
temp:=temp+1
ON
temp:=24
Off_cond
cond
minus
temp:=temp-1
OFF
inv-est
temp:=24
inv-est
temp:=20
OFF
Off_pom
pompa
minus
temp:=temp-1
ON
temp:=20
plus
temp:=temp+1

Esercizio

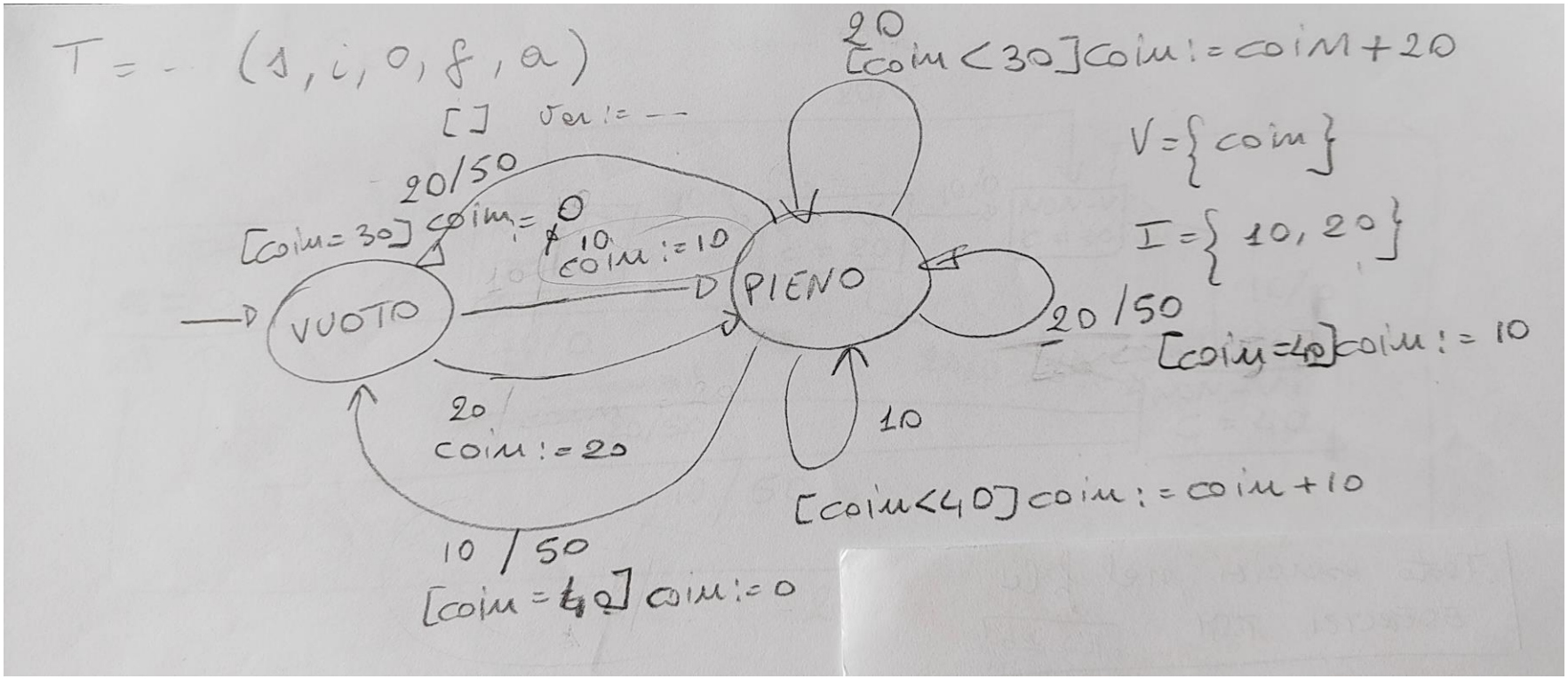
Modellare con una FSM un automa che fornisce monete da 50 centesimi in cambio di monete da 10 e 20 centesimi. La macchina non fornisce resto.

Soluzione diversa in base al formalismo di modellazione scelto e in base al livello di astrazione scelto

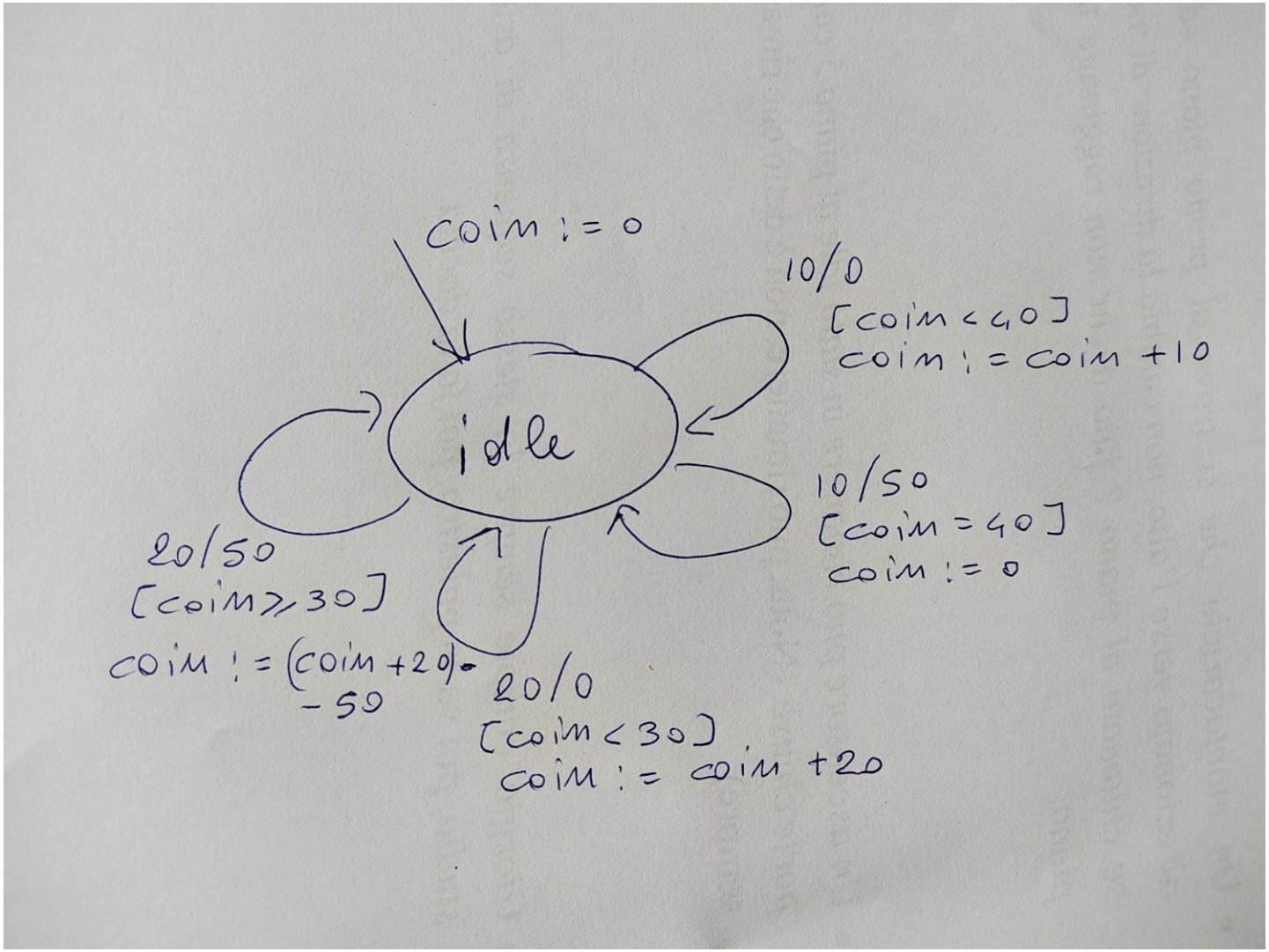
FSM I/O



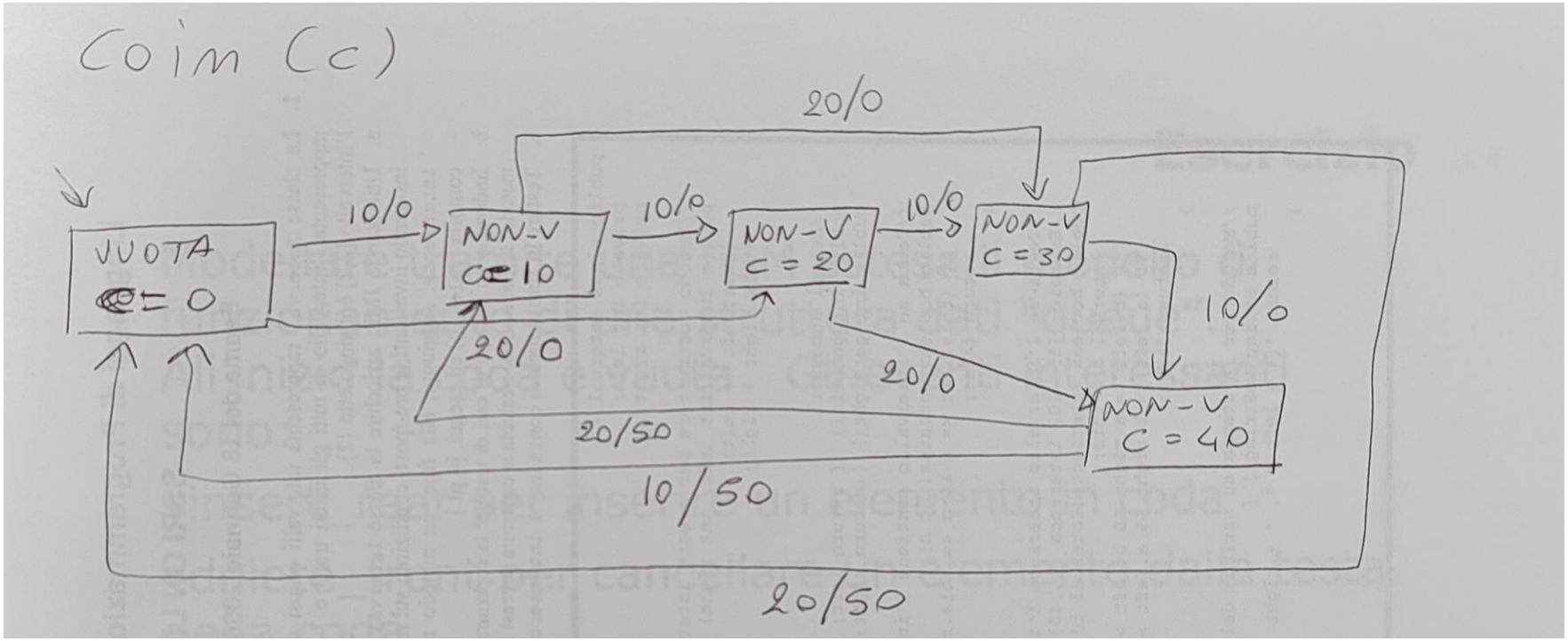
FSM estese 1


T = (s, i, o, f, a)
[] var := —
20/50
[coin = 30] coin := 0
10
coin := 10
VUOTO
PIENO
20
[coin < 30] coin := coin + 20
V = { coin }
I = { 10, 20 }
20/50
[coin = 40] coin := 10
20
coin := 20
10
[coin < 40] coin := coin + 10
10 / 50
[coin = 40] coin := 0

FSM estese 2


coin := 0
idle
10/0
[coin < 40]
coin := coin + 10
10/50
[coin = 40]
coin := 0
20/50
[coin ≥ 30]
coin := (coin + 20) - 50
20/0
[coin < 30]
coin := coin + 20

Trasformare EFSM in FSM I/O



Da ESFM soluzione 1

PER ESERCITARSI

Esercizio: FSM estese

Modellare con una macchina a stati finiti il ciclo di vita di un conto corrente in banca. Gli eventi interessanti sono:

- *deposit(m)* per depositare un certo ammontare *m* di denaro
- *withdraw(m)* per prelevare un certo ammontare *m* di denaro

E' sempre possibile effettuare prelievo di denaro, fintanto che il bilancio del conto non scende al di sotto di un certo limite *overdraft_limit.* Se si raggiunge tale limite, o se il bilancio si mantiene negativo per più di 3 mesi, il conto corrente viene "bloccato".

Esercizio : FSM estese

Modellare tramite una FSM estesa il modello di funzionamento di una struttura dati “queue”. All’inizio la coda è vuota. Gli eventi interessanti sono:

- insert_rear per inserire un elemento in coda
- delete_front per cancellare un elemento dalla testa della coda

Una variabile globale size rappresenta l’effettiva lunghezza della coda. Essa vale 0 quando la coda è vuota, e k (una costante globale) quando è piena.

Esercizio : FSM estese

Utilizzando una macchina a stati finiti estesa modellate il comportamento del seguente distributore di bevande. Un distributore automatico di bevande consente l'acquisto di una bevanda a scelta mediante apposita chiavetta pre-pagata. Il dispositivo permette all'utente di selezionare la bevanda desiderata tramite la digitazione del codice della bevanda su una tastiera di cui il distributore è fornito.

Se la bevanda non è disponibile, tramite display viene visualizzato un opportuno messaggio che la bevanda non è erogabile. Se la bevanda è disponibile, sul display viene visualizzato il costo della bevanda. Se l'utente inserisce la chiavetta pre-pagata entro il tempo massimo di 1 minuto, il costo della bevanda viene sottratto all'ammontare di soldi contenuti nella chiavetta e la bevanda viene erogata. Se la carica della chiavetta non riesce a coprire il costo della bevanda,viene visualizzato un messaggio che l'erogazione non è possibile per ammontare insufficiente. Se l'utente non inserisce la chiavetta entro il tempo massimo stabilito, la scelta viene annullata ed il distributore ritorna nello stato iniziale di pronto.

RIPRENDIAMO….

MACCHINE DI COMUNICAZIONE

Limiti delle FSM

Con le FSM è possibile rappresentare solo un numero finito di stati e componendo diverse FSM si ha il problema della

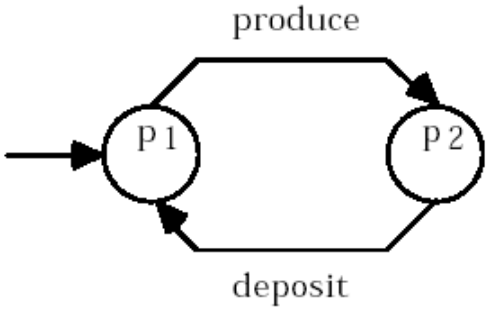
Esplosione del numero di stati:

- dato un numero di FSM con k1, k2, … kn stati ciascuna, la loro composizione è una FSM con k1 * k2 *… * kn stati
- tale crescita è esponenziale nel numero di FSM

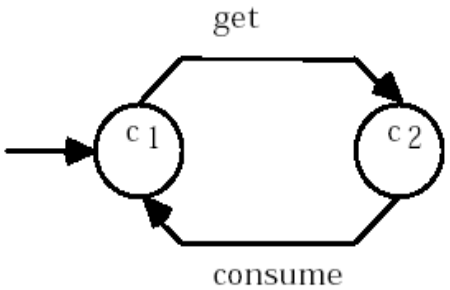
ci piacerebbe una crescita lineare, ossia k1 + k2 +… + kn stati

Esplosione degli stati: esempio

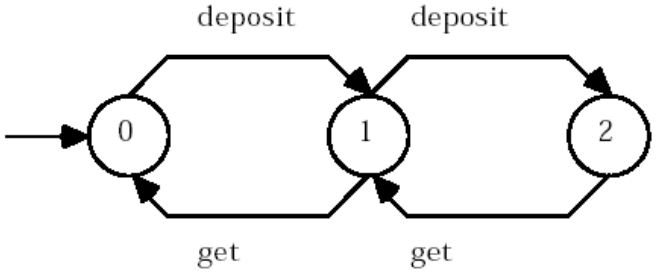
Produttore



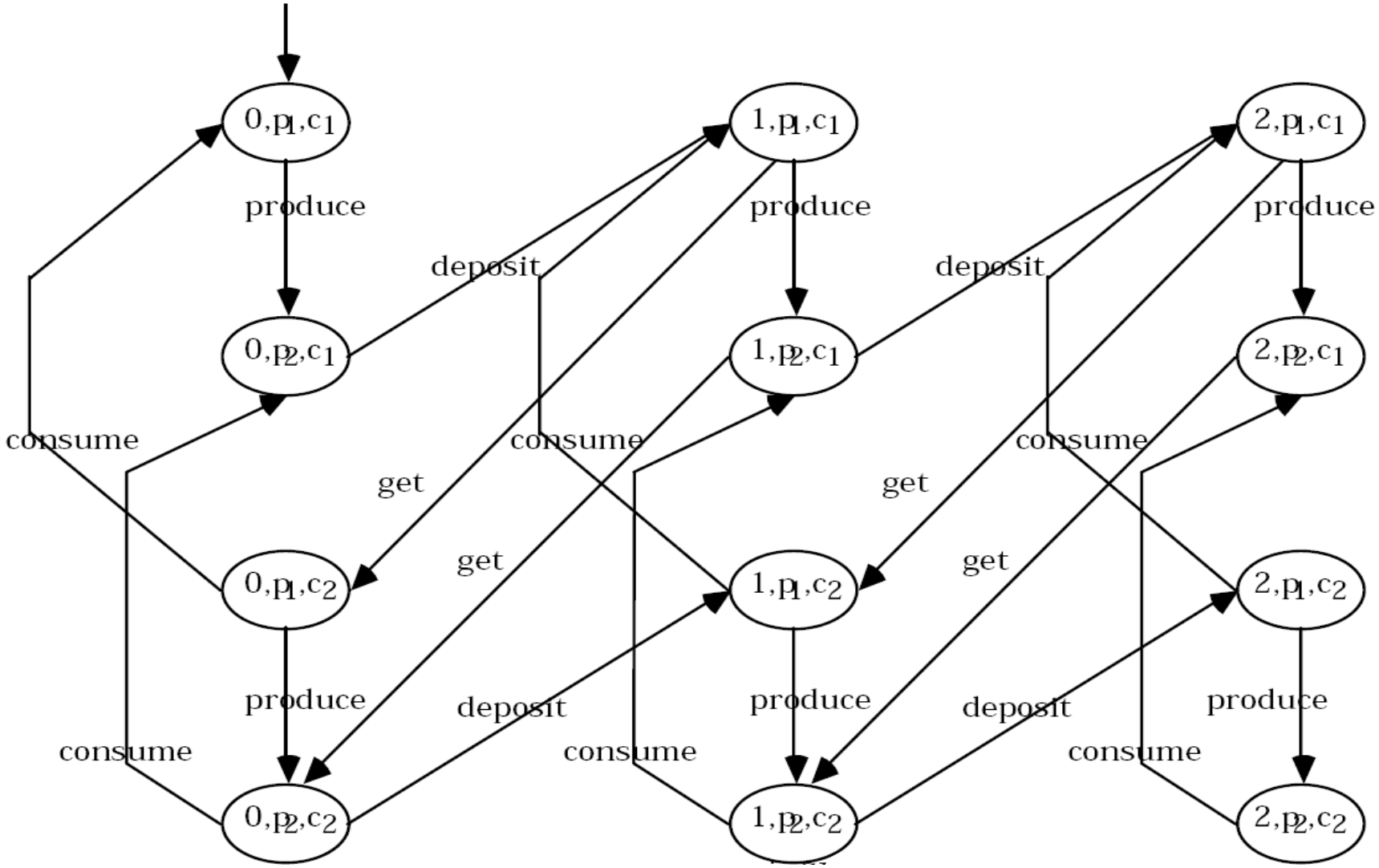
Consumatore



Magazzino



Combinando le FSM



Limiti delle FSM

Per superare i limiti di composizionalità delle FSM, sono state definite opportune estensioni (tra cui le Statecharts di UML) che sono dotate dei concetti di sottomacchina e permettono

- Composizione sequenziale
- Composizione parallela
- Modularità

Communicating Finite State Machines (1)

- Alcuni sistemi possono essere modellati in maniera più semplice con un insieme di FSM che operano in modo concorrente ed interagiscono scambiandosi messaggi, piuttosto che con una singola FSM

- Le *CFSM (Communicating Finite State Machines)* modellano un insieme di FSM concorrenti che interagiscono tra loro

- Le CFSMs sono particolarmente utilizzate per la specifica di sistemi embedded e protocolli di comunicazione

Communicating Finite State Machines (2)

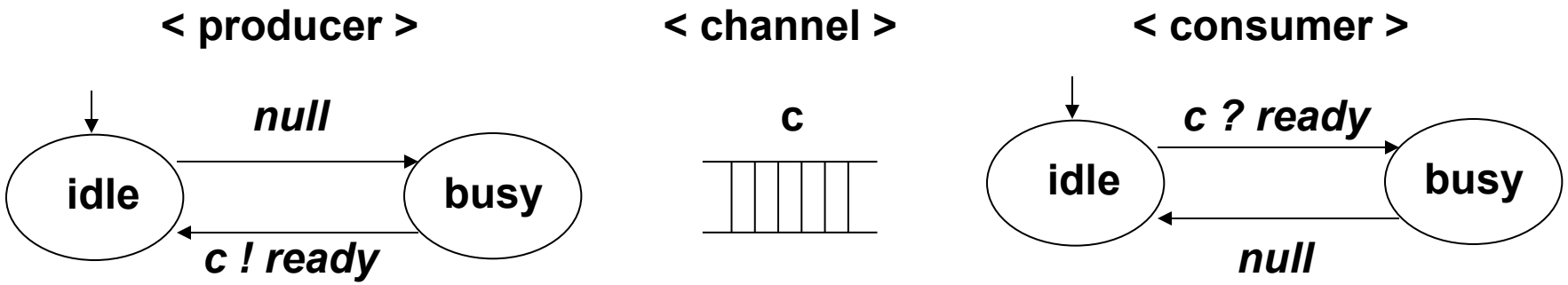
Una CFSM è una coppia (C, P) dove

- C è un insieme finito di canali (message queues)
- P è un insieme finito di processi
  - Un processo è una tupla (S, I, O, T)
    - S : insieme finito di stati
    - I : insieme finito di input
    - O: insieme finito di output
    - T : insieme finito di transizioni della forma
      (s, null, s'), (s, c?i, s'), (s, c!o, s')

Communicating Finite State Machines (3)

Esempio

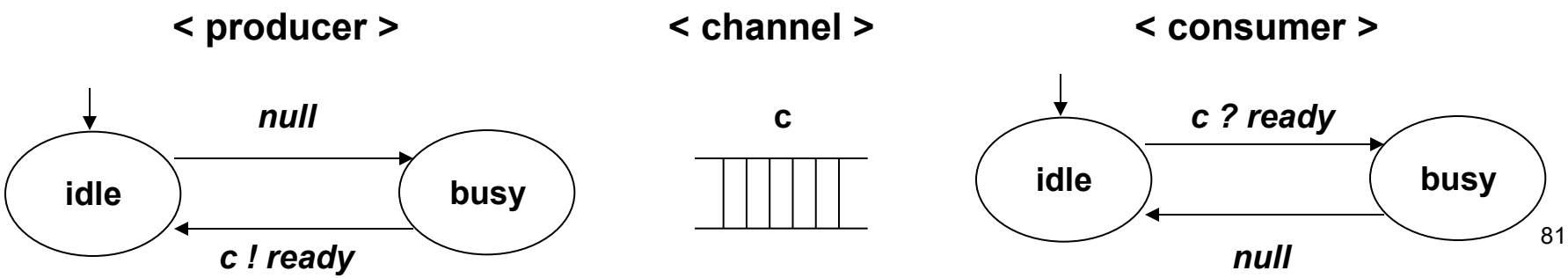
Un produttore ed un consumatore comunicano attraverso un canale



Communicating Finite State Machines (4)

Esempio

- C = {c}
- P = {producer, consumer}
  - producer = (S, I, O, T)
    - S = {idle, busy}, I = {}, O={ready}
    - T = {(idle,null,busy), (busy,c!ready,idle)}
  - consumer = (S′, I′, O′, T′)
    - S' = {idle, busy}, I' = {ready}, O' ={}
    - T' = {(idle, c?ready,busy), (busy,null,idle)}



Estensione delle CFSM

Una CFSM estesa è una coppia (C, P) dove

- C è un insieme finito di canali
- P è un insieme finito di processi
  - Un processo è una tupla (S, I, O, V, T)
    - S : insieme finito di stati
    - I : insieme finito di input
    - O: insieme finito di output
    - V: insieme finito di variabili
    - T : insieme finito di transizioni della forma

      (s, g, a, s'), (s, g, c?i, s'), (s, g, c!o, s')

      con

      g : predicato sulle variabili in V, detto **guardia**

      a : assegnamento di variabili in V, detto **azione**

Estensione temporale delle CFSM

Una CFSM temporale è una CFSM estesa in cui le transizioni dei processi possono contenere informazione dell'intervallo temporale entro cui l'azione può avvenire dopo essere entrati nello stato

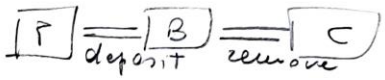
- L'insieme delle transizioni di un processo può essere della forma (s, g, a|I/O, s', [t1,t2])
  - [t1,t2] indica l'intervallo di tempo entro cui il processo può eseguire un' azione interna a o una azione di I/O (c?i, c!o) dopo che è entrato in s prima di raggiungere s'

Esercizio

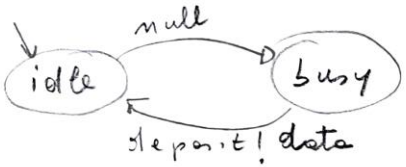
Modellare con il formalismo delle Communicating FSM estese il comportamento di un sistema producer/consumer con buffer di memoria. Il buffer è anch'esso visto come processo che interagisce con le macchine producer e consumer attraverso due canali rispettivamente deposit e remove. L'attività interna del buffer è di tipo computazionale. Esso contiene una variabile vettoriale buf di capacità limitata da 0 a n-1 (con n una costante) usata come buffer circolare per contenere i dati. La variabile full mantiene l'effettiva lunghezza del buffer. Due variabili in e out fanno da indici alla variabile buf: buf[in] è l'elemento ricevuto in input dal producer, mentre buf[out] è l'elemento prodotto in output per essere consumato dal consumer. Inizialmente in=out=full=0.

Modello in CFSL estese

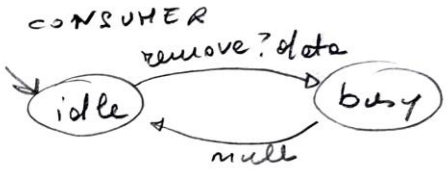
C = { deposit, remove }
P = { producer, consumer, buffer }



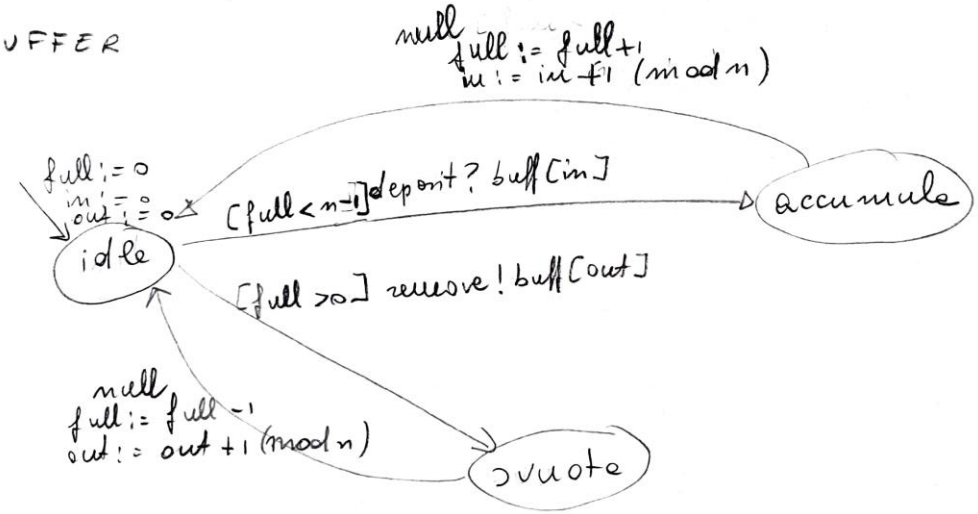
PRODUCER



CONSUMER



BUFFER



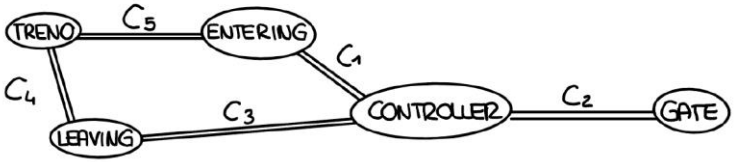
n : costante
n-1 : capienza max del buffer

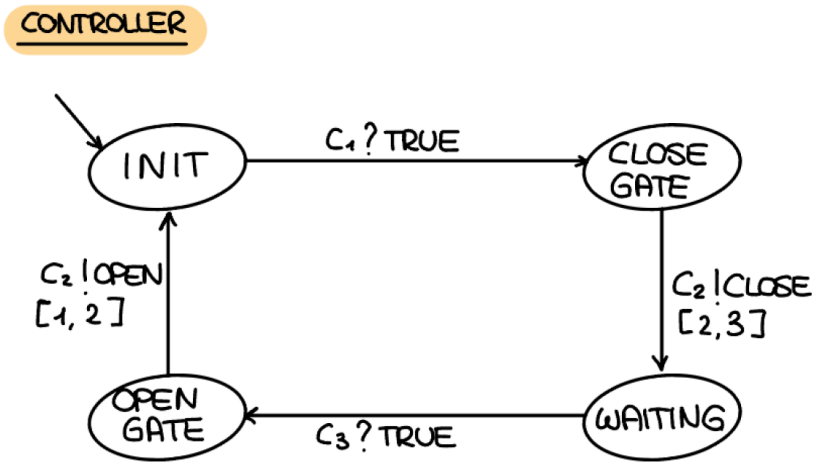
Esercizio

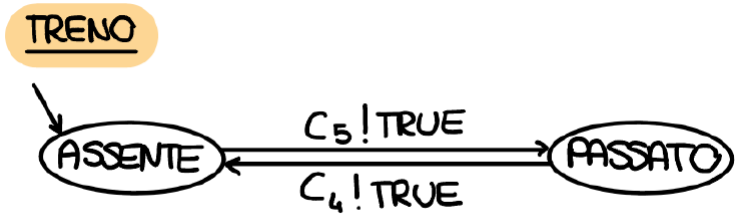
Una linea tranviaria unidirezionale incrocia una strada. Due sensori ed un computer controllano la sbarra. Un sensore rileva l'arrivo del treno, mentre l'altro rileva l'avvenuto passaggio.

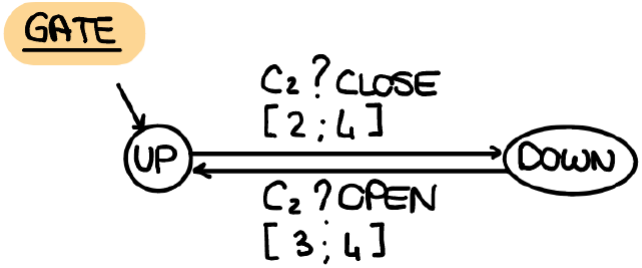
Modellare con il formalismo delle Communicating Machine estese e temporali 5 macchine: una per il treno (Train), 2 per i relativi sensori (Entering-Sensor, Leaving-Sensor), una per la sbarra (Gate), ed una per il sistema di controllo (Controller) vero e proprio. Quest'ultimo, su segnalazione del sensore Entering-Sensor, impiega da 2 a 3 tick di tempo per inviare alla sbarra il segnale di abbassarsi e poi aspetta un segnale dal sensore Leaving-Sensor. La sbarra impiega da 2 a 4 tick per raggiungere la posizione down. Dopo che il treno è passato, il Leaving-Sensor invia un segnale al controllore che il treno è passato. Ricevuto quest'ultimo segnale, il sistema di controllo impiega da 1 a 2 tick per inviare il segnale alla sbarra per rialzarla. Infine, la sbarra prende da 3 a 4 tick per rialzarsi definitivamente.

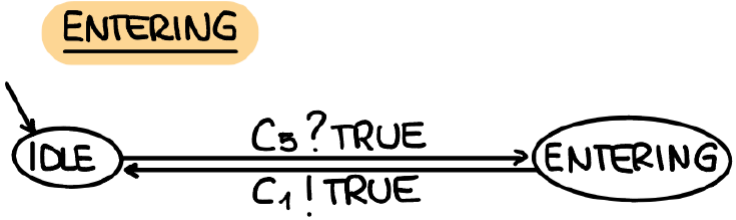
Modello in CFSM temporali

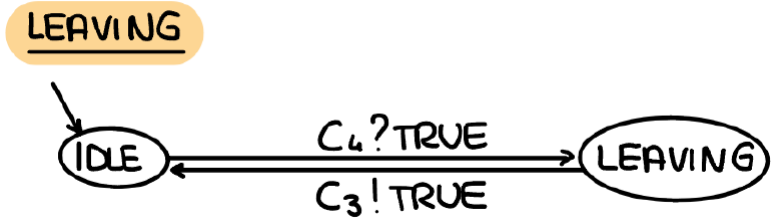












PER ESERCITARSI

Esercizio

Modellare con il formalismo delle Communicating FSM estese il comportamento di un sistema producer/consumer con buffer di memoria.

Il buffer è anch'esso visto come processo che interagisce con le macchine producer e consumer attraverso due canali rispettivamente *deposit* e *remove*.

L'attività interna del buffer è di tipo computazionale. Esso contiene una variabile vettoriale buf di capacità limitata da 0 a n-1 (con n una costante) usata come buffer circolare per contenere i dati. La variabile *full* mantiene l'effettiva lunghezza del buffer. Due variabili *in* e *out* fanno da indici alla variabile *buf*: buf[in] è l'elemento ricevuto in input dal producer, mentre buf[out] è l'elemento prodotto in output per essere consumato dal consumer. Inizialmente *in=out=full=0*.

Esercizo

Modellare con una macchina di comunicazione temporale il comportamento del seguente sistema di controllo di due sbarre. Il sistema di controllo è costituito da due sbarre collegate ad un computer che, per ciascuna sbarra, ne determina la posizione di aperta (UP) o chiusa (DOWN). Il computer controlla la sequenza di apertura/chiusura delle sbarre emettendo i segnali *open* (= vai a UP) e *closed* (= vai a DOWN) a cui le sbarre rispondono cambiando posizione. Il funzionamento delle sbarre è controllato dal computer in base al seguente ciclo fisso di quattro fasi: fase1, per 50 secondi, entrambe le sbarre sono in DOWN; fase2, per 100 secondi una sbarra è in UP e l'altra è in DOWN; fase3, per 50 secondi entrambe le spie sono in DOWN nuovamente; fase4, per 100 secondi la sbarra che prima era in UP passa a DOWN mentre l'altra passa a DOWN ad UP. Ed il ciclo si ripete.

RIPRENDIAMO….

MACCHINE DI STATO UML

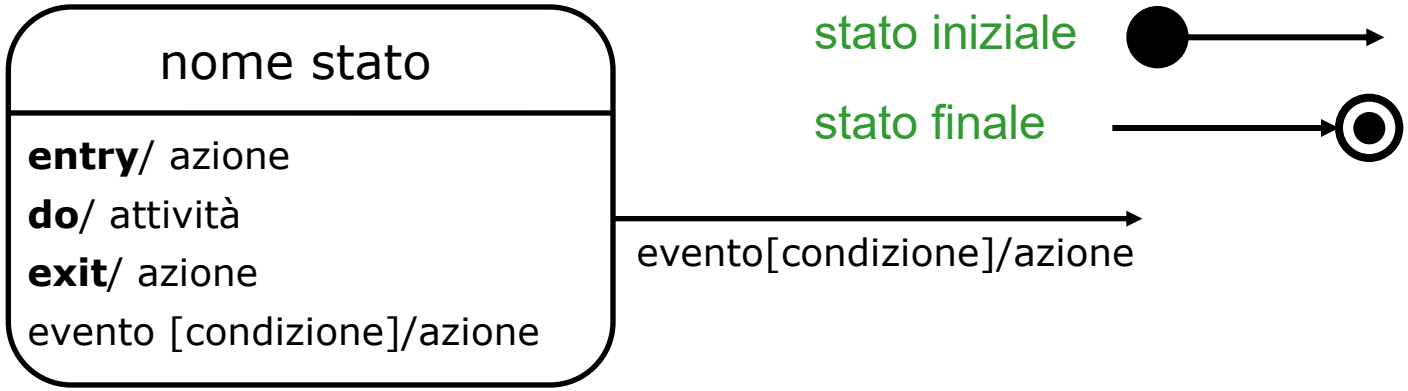
Macchina di stato

- E' solo uno dei tipi di diagrammi UML che consente la modellazione dinamica
- E' associata ad una classe e descrive il comportamento di un oggetto della classe
- UML adotta la notazione di Harel
- E' un grafo i cui nodi rappresentano gli stati in cui può trovarsi un oggetto ed i cui archi rappresentano le transizioni tra gli stati

Stato

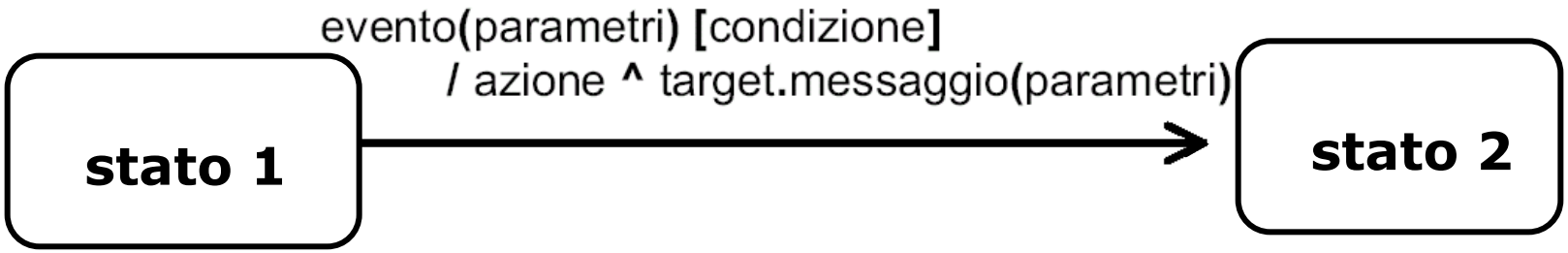
Uno stato rappresenta una condizione di esistenza dell' oggetto:

- attesa di un evento
- esecuzione di certe azioni o attività
  - **Azioni**: operazioni atomiche e non interrompibili
    - entry: azione eseguita entrando nello stato
    - exit: azione eseguita uscendo dallo stato
  - **Attività**: operazioni che richiedono un certo tempo



Transizione (1)

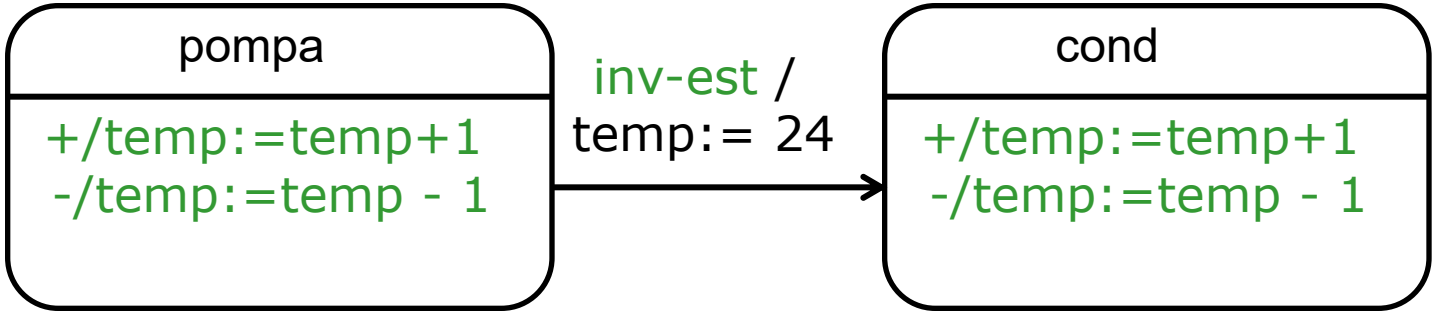
- Una **transizione** indica un passaggio di stato



- E' **etichettata** con le seguenti informazioni:
  - gli eventi che comportano il cambiamento di stato e le condizioni sotto cui ha effetto
  - le azioni che l'oggetto esegue prima di cambiare stato

Transizione (2)

- Una transizione **scatta** quando occorre un evento, *interno* o *esterno,* e l' eventuale guardia sulla transizione è vera



- In UML si assume che:
  - si verifica una sola istanza di evento alla volta
  - l' istanza di un evento non ha durata

Tipi di evento

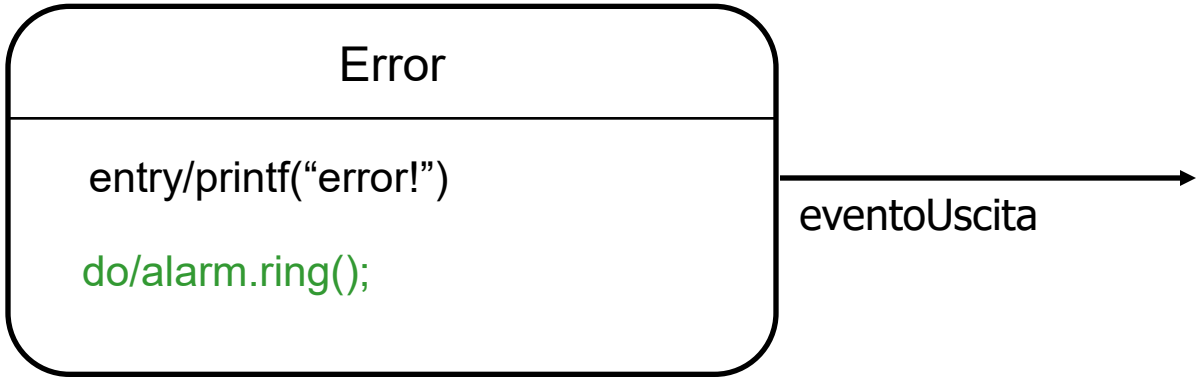
- **Interazione con altri oggetti**
  - Call event: si verifica nel momento in cui un oggetto invoca un'operazione propria o di un altro oggetto
  - Signal event: (**<<signal>>** e) si verifica nel momento in cui un oggetto riceve un *segnale* da un altro oggetto
- **Occorrenza di instanti di tempo**
  - Time event: intervallo temporale, data o clock di tempo
    - **after** seguita da quantità temporale (es. **after**(10 sec)
- **Cambiamento del valore**
  - Change event: indicato con **when**(espr. booleana)

    (es. **when**(bilancio<0))
    - L'espressione viene controllata quando variano i parametri coinvolti

Attività di stato

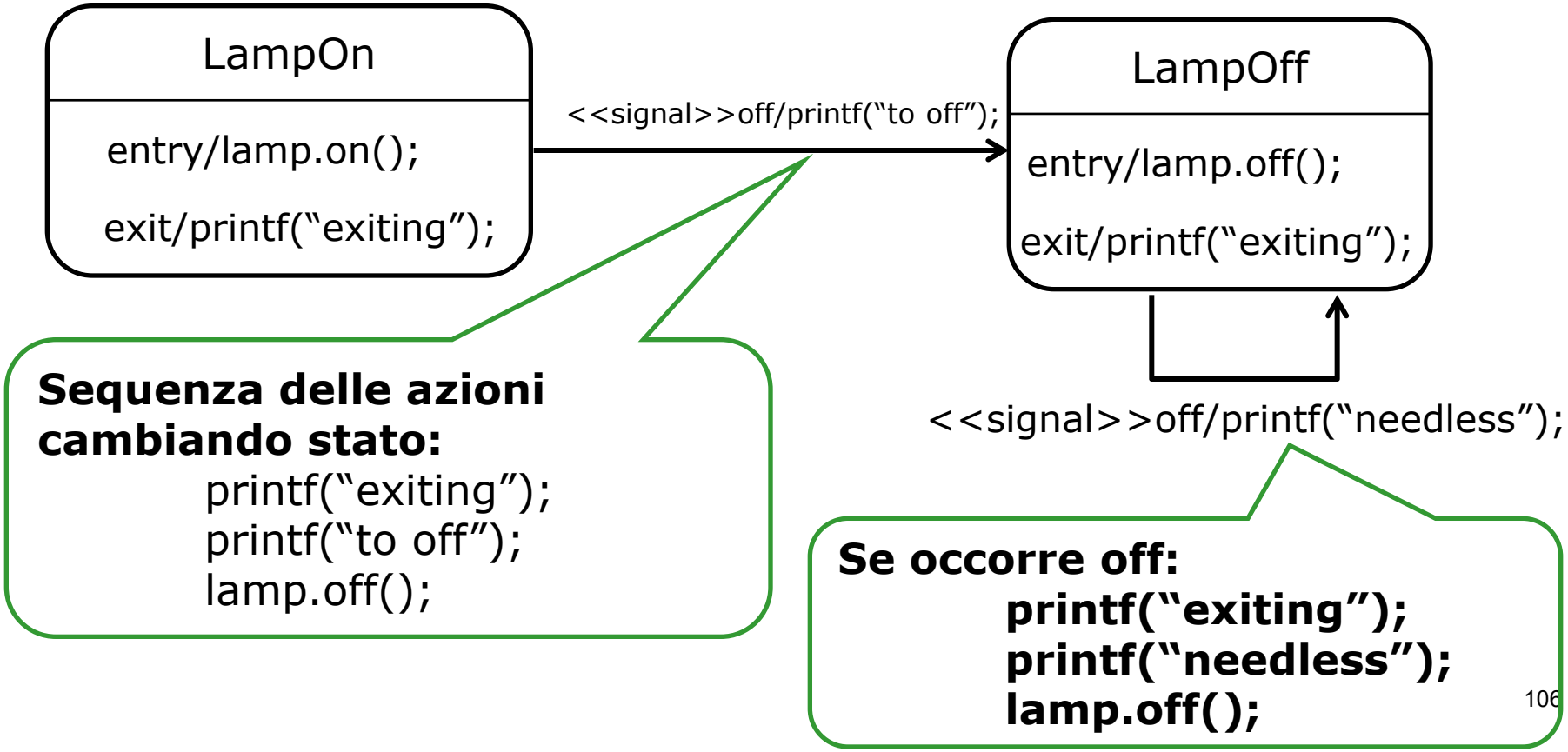
Attiva un thread concorrente che esegue finché:

- attività è completata, o
- transizione di uscita abilitata



Ordine delle azioni: caso 1

- Azioni **exit** **precedono** quelle della transizione
- Azioni **entry** **seguono** quelle della transizione



Esercizio

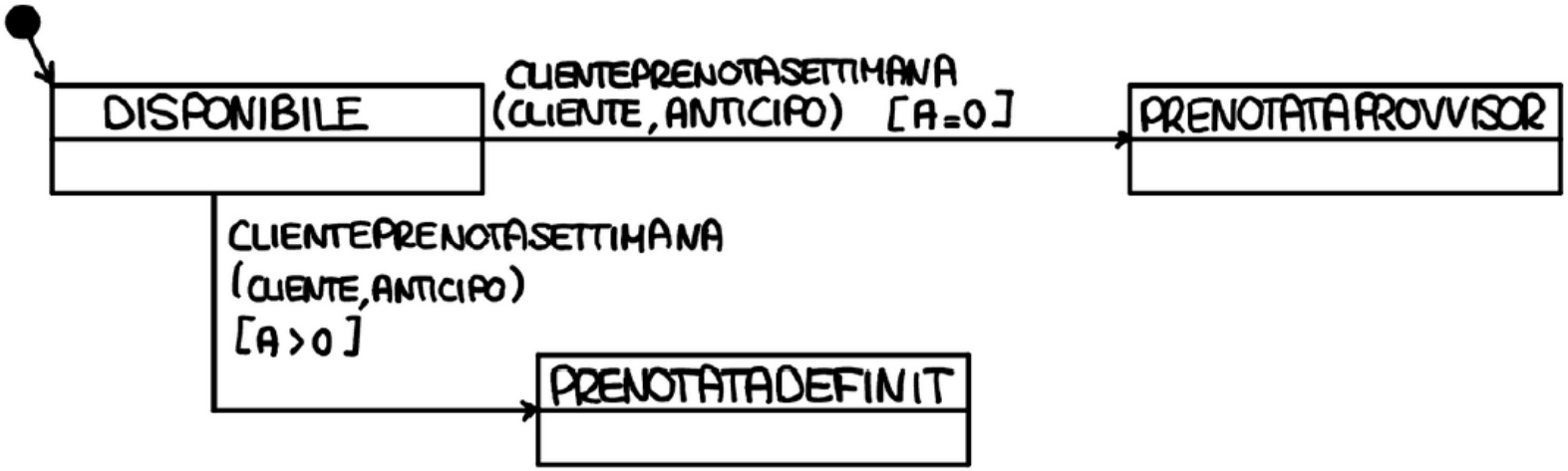
Una piccola società S.r.l. di formazione offre un seminario a settimana. Ogni settimana del calendario di insegnamento è un'istanza della classe *SettimanaSeminario*. Lo stato di prenotazione di una data settimana può essere *disponibile*, *prenotataProvvisoria* e *prenotataDefinitiva*.

Quando viene creata un'istanza di SettimanaSeminario (ossia viene collocata sul calendario), lo stato iniziale è disponibile. Un cliente può prenotare una settimana con o senza il versamento di un anticipo (tramite l'operazione clientePrenotaSettimana(cliente,anticipo)).

In caso di anticipo, il cliente ha diritto ad una prenotazione confermata (prenotataDefinitiva) altrimenti può ottenere solo una prenotazione provvisoria.

Disegnare la porzione di diagramma degli stati UML per i tre stati di SettimanaSeminario appena descritti.

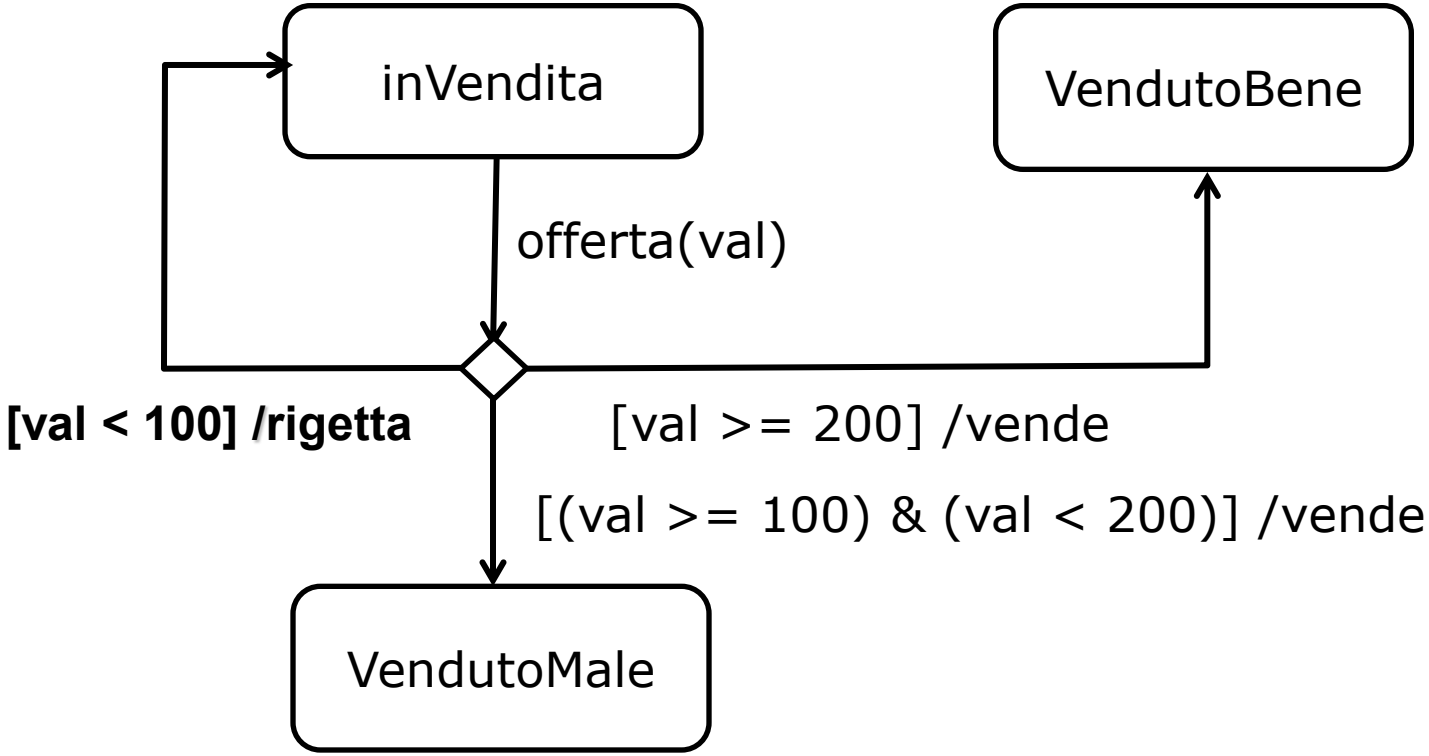
Modello


DISPONIBILE
CLIENTEPRENOTASETTIMANA
(CLIENTE, ANTICIPO) [A=0]
PRENOTATAPROVVISOR
CLIENTEPRENOTASETTIMANA
(CLIENTE, ANTICIPO)
[A>0]
PRENOTATADEFINIT

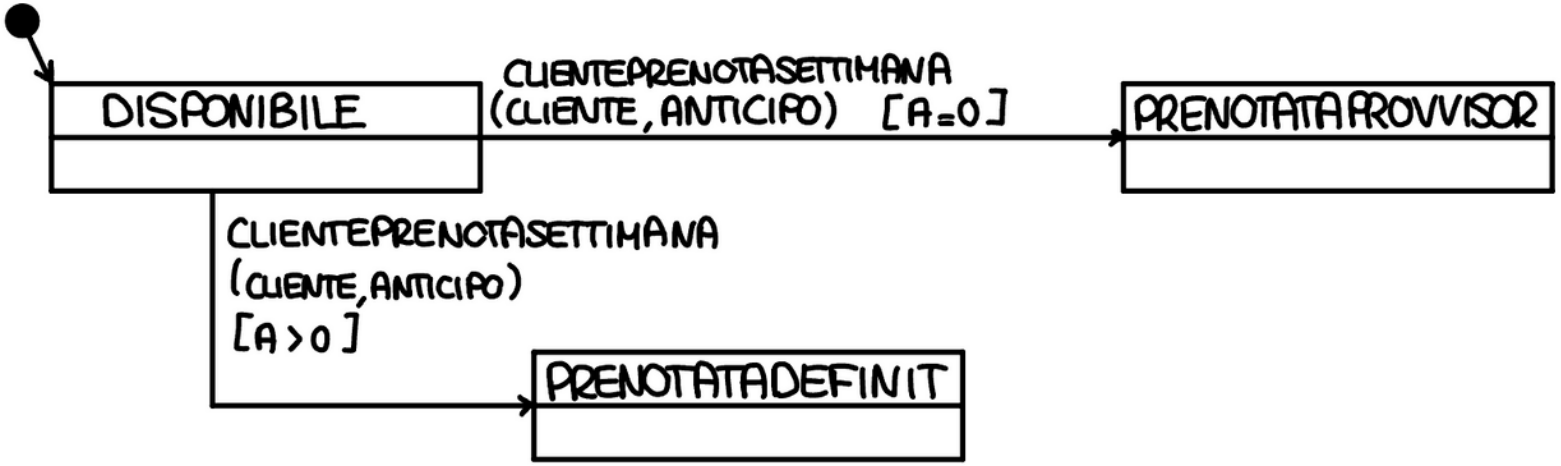
Dynamic Conditional Branching

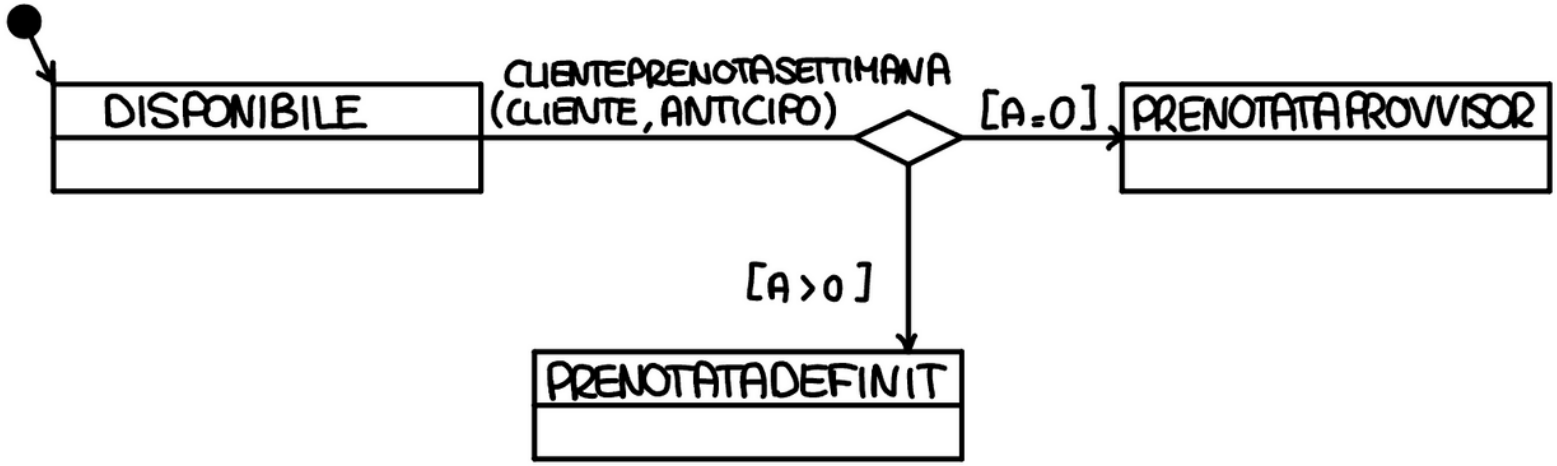
Choice pseudostate: le guardie sono valutate nell'istante in cui il punto di decisione è raggiunto:

- se più di una decisione è vera, si selezione 1 transizione random;
- se nessuna è vera il modello non è corretto



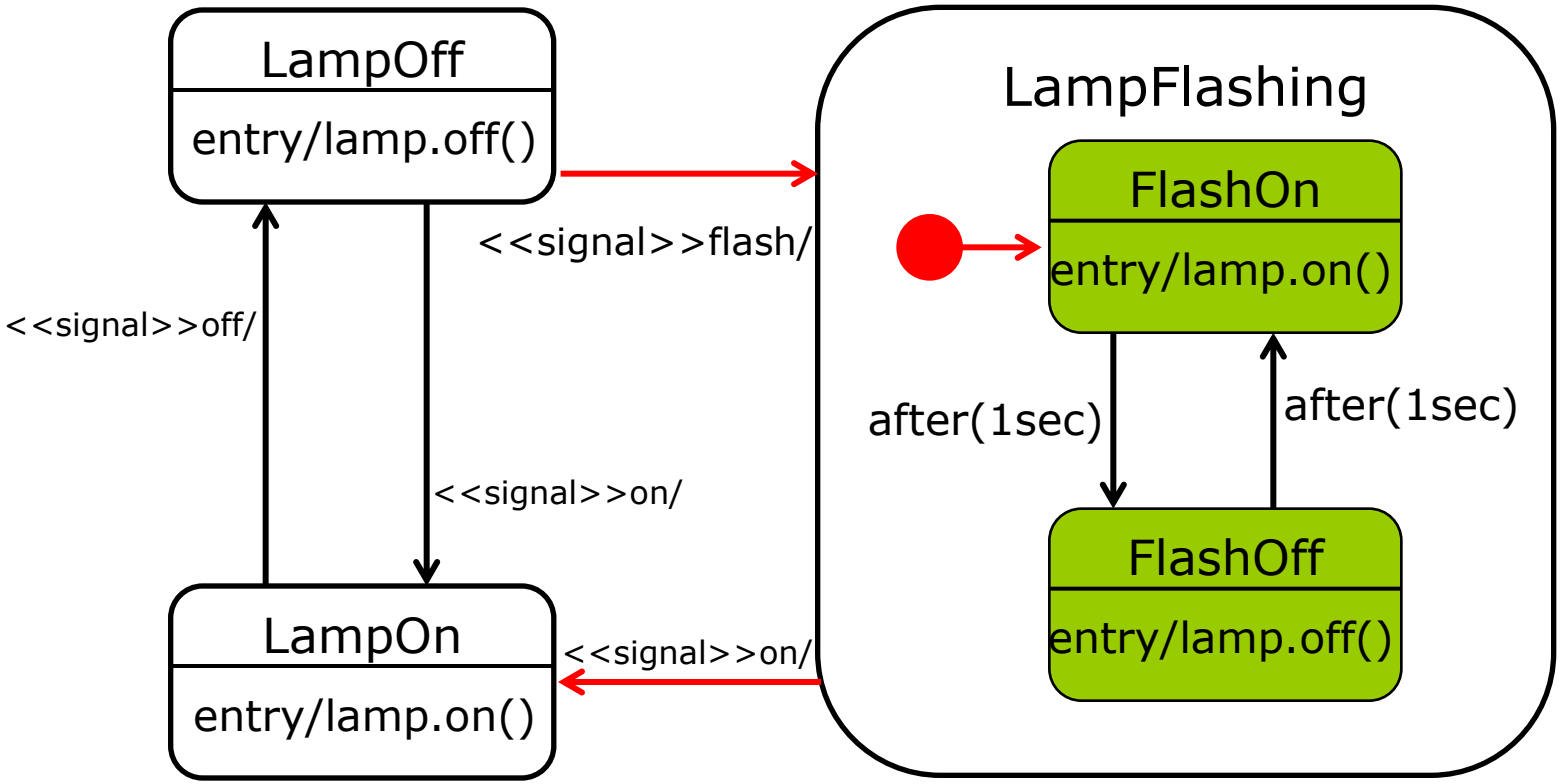
Modello rivisto



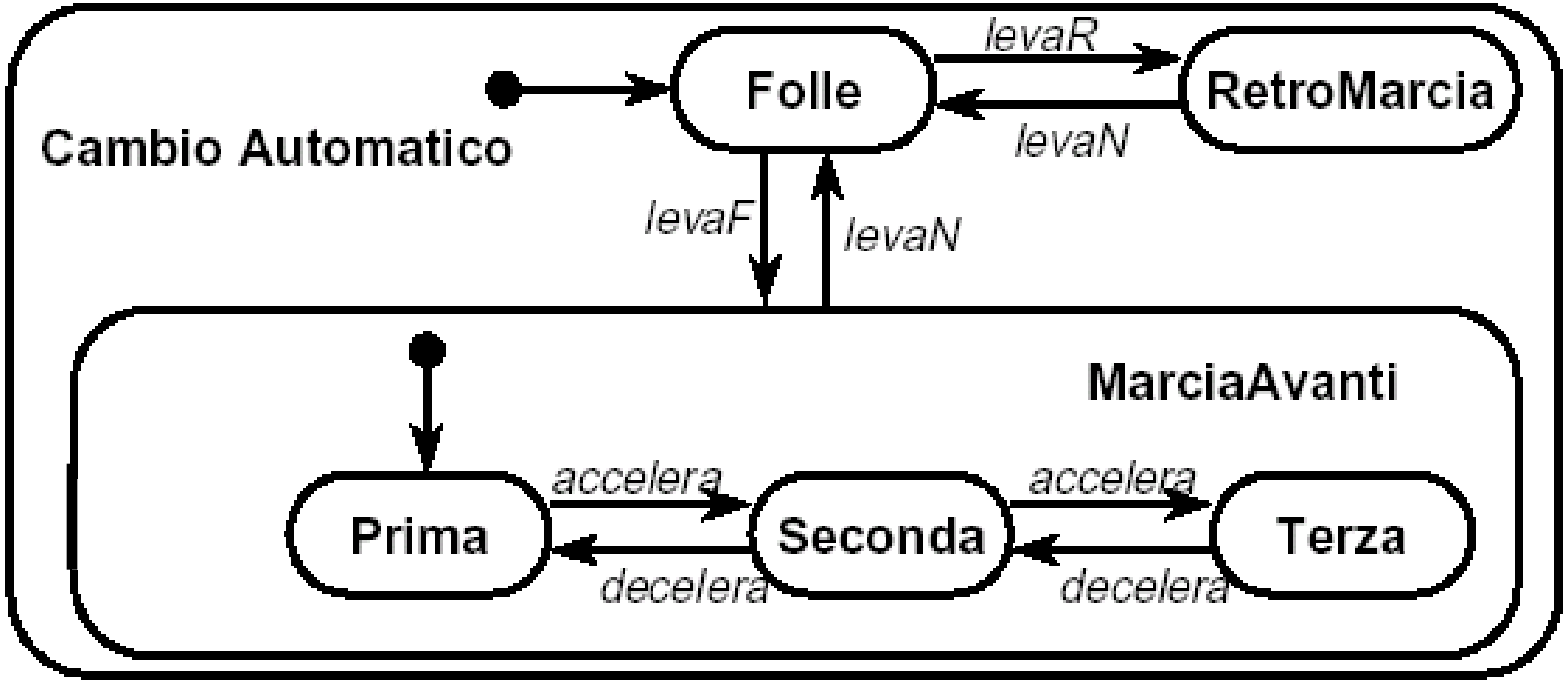


Stati composti sequenziali

- Uno stato può contenere una macchina di stato



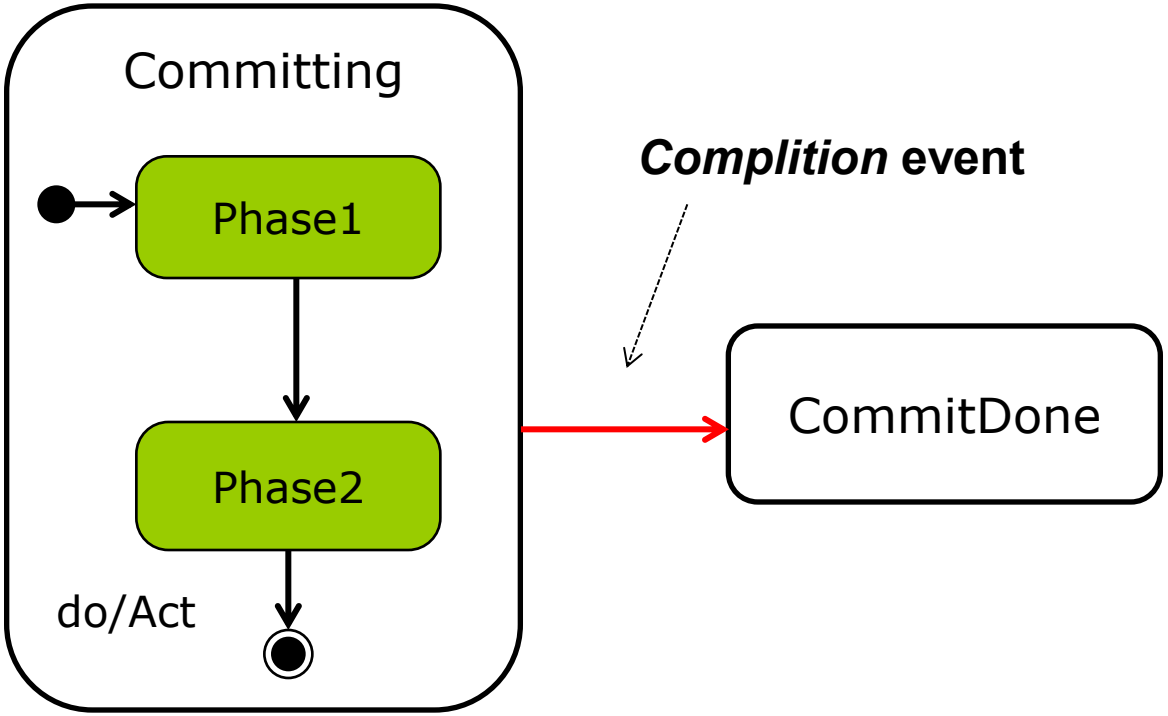
Esempio: cambio automatico



Transizione di completamento

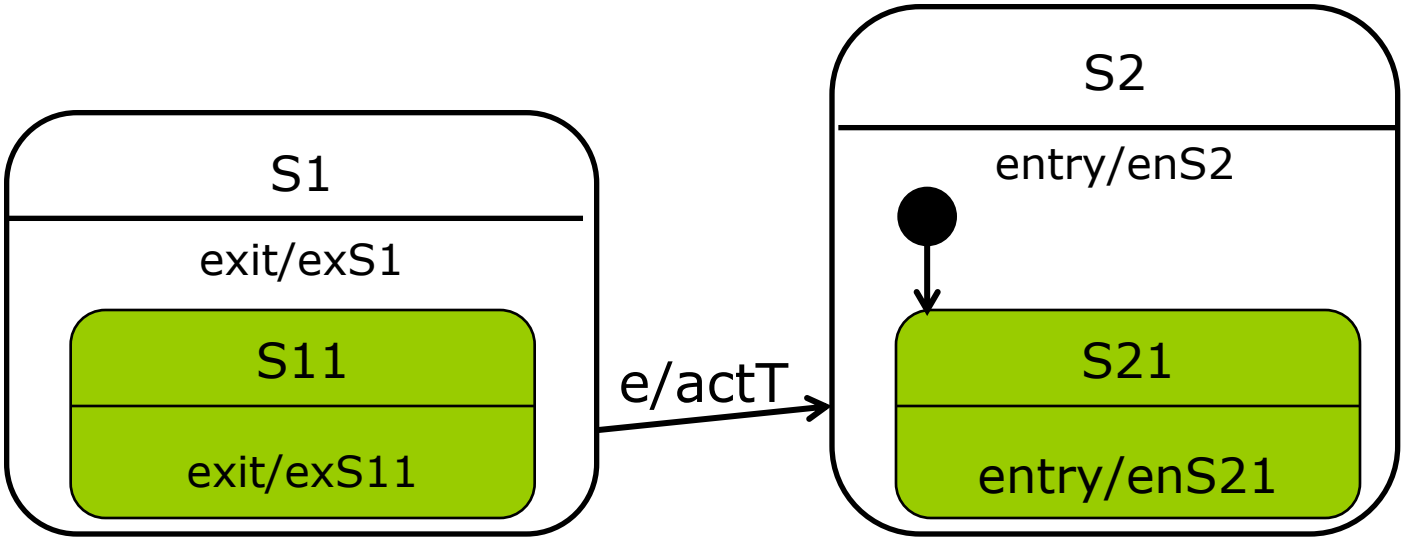
Attivata da un completion event

- Generato quando una macchina annidata termina o una attività interna termina



Ordine delle azioni: caso 2

- Stesso approccio come in caso 1

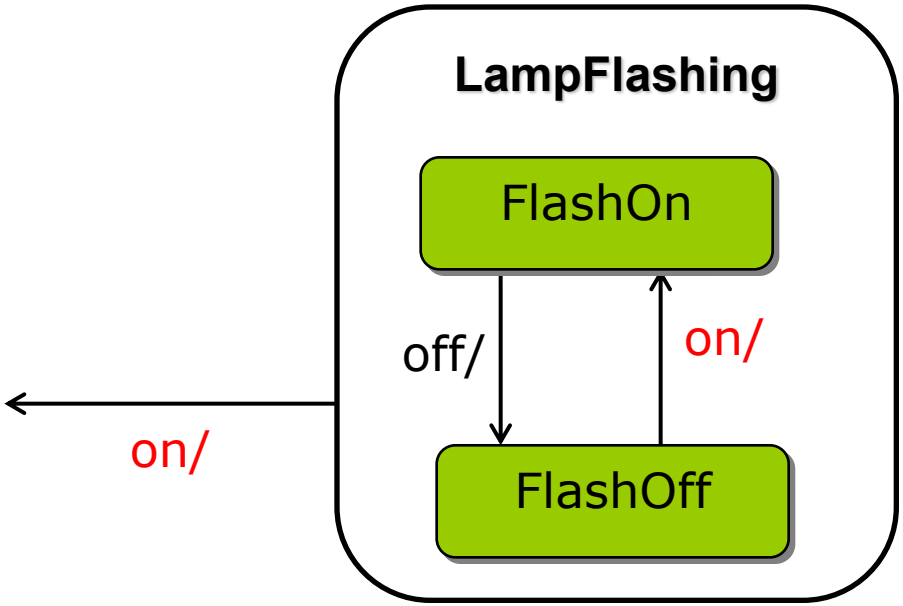


- Sequenza di esecuzione delle azioni:

  exS11 ⇨ exS1 ⇨ actT ⇨ enS2 ⇨ enS21

Regole di scatto

- Due o più transizioni possono essere abilitate dallo stesso evento nello stesso istante
- Hanno la precedenza le transizioni più interne



Esercizio

Modellare in UML una macchina di stato relativa al ciclo di vita di un applet Java. Un applet è un'applicazione Java progettata per essere eseguita sul Web. Quando un browser web visualizza una pagina (tipo html) che contiene un applet, insieme agli altri elementi della pagina viene caricato (istanziato) l'oggetto applet. I metodi dell'applet attraverso cui il browser gestisce un oggetto applet sono i seguenti:

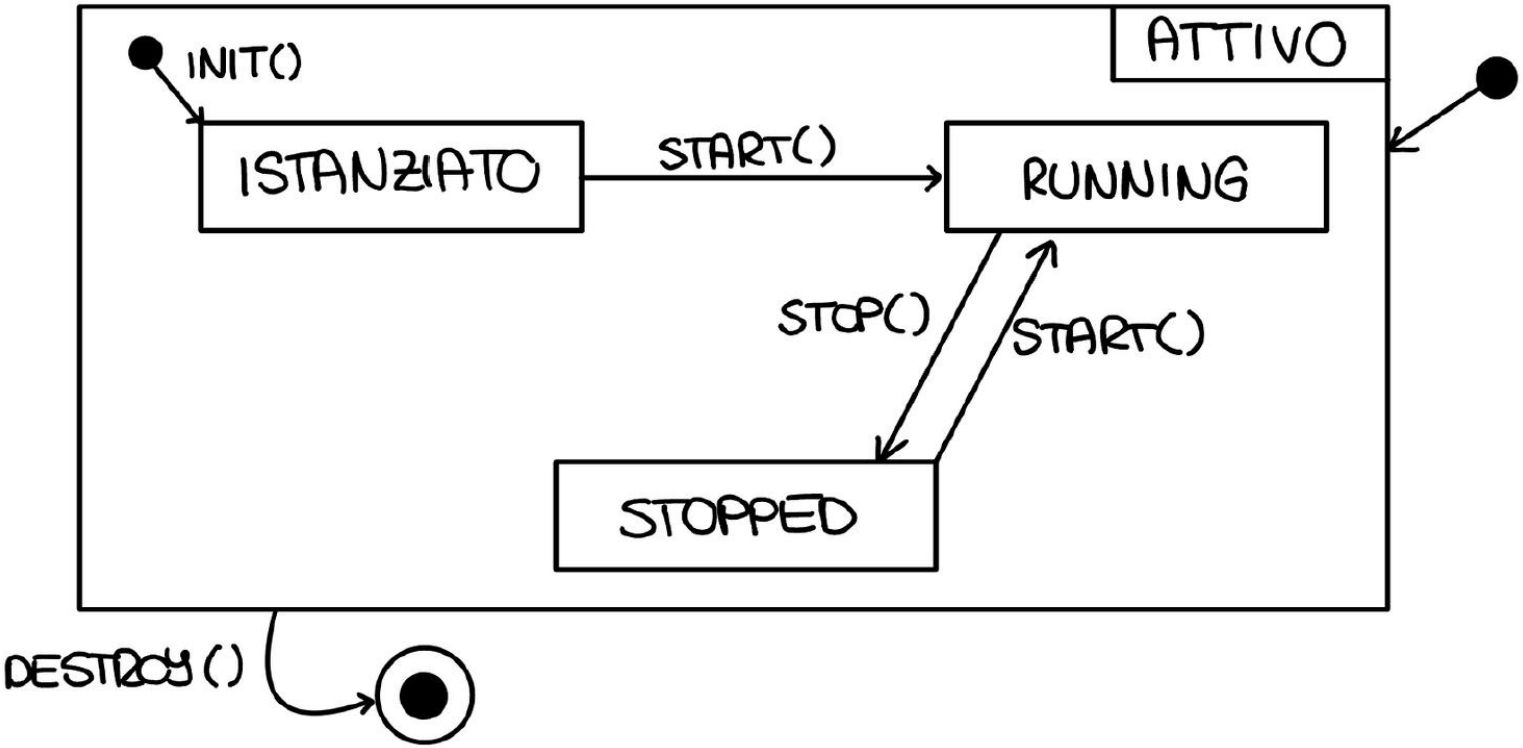
init() : chiamato quando l'applet viene caricato nella finestra del browser;

start(): chiamato per avviare l'esecuzione dell'applet all'inizio e ogni qual volta l'applet da uno stato in cui ne viene arrestata l'esecuzione (stopped) passa ad uno stato attivo (running);

stop(): chiamato per arrestare l'esecuzione dell'applet, ad esempio, quando l'utente lascia momentaneamente la pagina web dove l'applet sta girando;

destroy(): chiamato per la rimozione dell'applet da parte del browser quando, per esempio, l'applicazione browser stessa viene chiusa.

Modello


ATTIVO
INIT()
ISTANZIATO
START()
RUNNING
STOP()
START()
STOPPED
DESTROY()

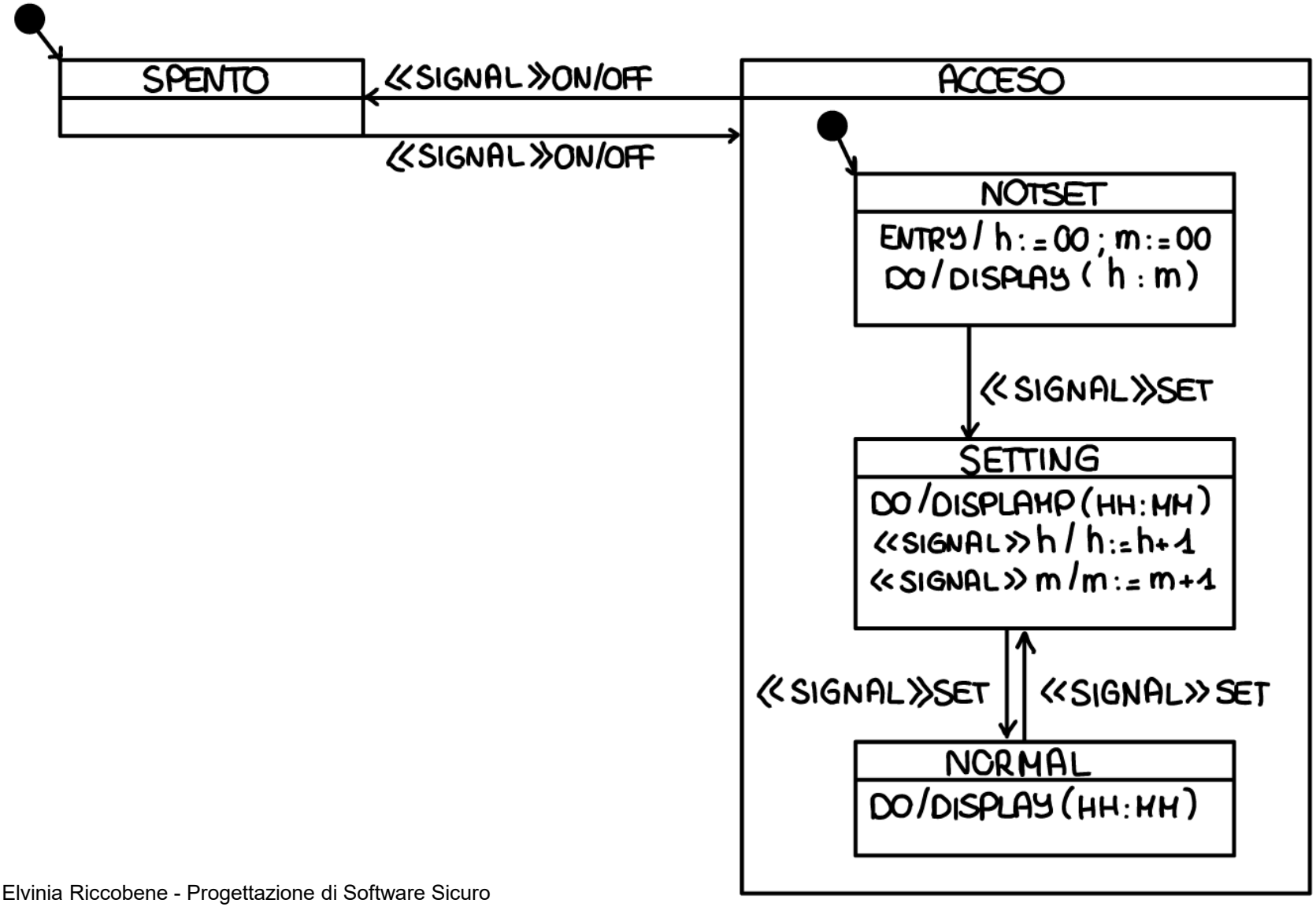
Esercizio

Modellare tramite una macchina di stato UML il funzionamento di un orologio digitale che segnala l'ora come hh:mm.

Un interruttore ON/OFF permette di accendere e spegnere l'orologio. Quando viene acceso, il display dell'orologio segnala un orario non attendibile come 00:00.

Il tasto set permette di aggiustare l'ora mediante i tasti m ed h per minuti ed ore; durante la fase di aggiustamento, il display segnala l'ora in modo lampeggiante. Settata l'ora, tramite set, l'orologio passa in modalità normale in cui l'orario viene visualizzato in modo non lampeggiante.

Modello


SPENTO
«SIGNAL»ON/OFF
«SIGNAL»ON/OFF
ACCESO
NOTSET
ENTRY / h:=00 ; m:=00
DO / DISPLAY (h : m)
«SIGNAL»SET
SETTING
DO / DISPLAMP (HH:MM)
«SIGNAL» h / h:=h+1
«SIGNAL» m / m:= m+1
«SIGNAL»SET
«SIGNAL» SET
NORMAL
DO / DISPLAY (HH:MM)

Esercizio

Un sistema ISPDialer può connettersi ad un internet service provider (ISP). Il sistema può quindi essere connesso, non connesso o trovarsi nella fase in cui cerca la connessione.

Durante il tentativo di connessione, il sistema mette inizialmente il modem in modalità off-hook ed aspetta il segnale di linea libera (dialtone).

Se il segnale dialtone non arriva entro 20 secondi, il sistema segnala la mancanza di linea libera e passa in modo non connesso rimettendo il modem in modalità hook. Il sistema viene quindi fermato.

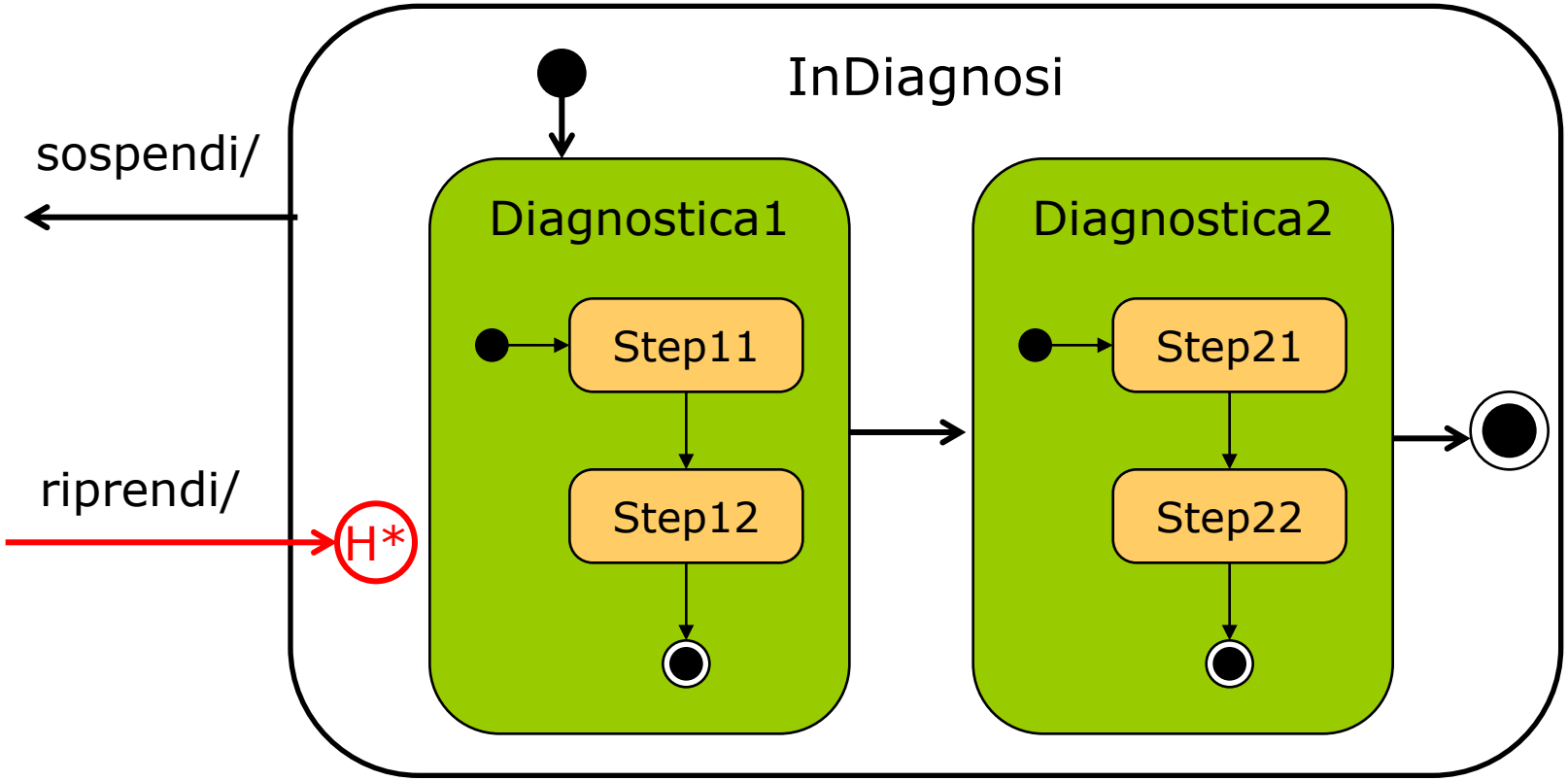
Se il segnale di linea libera viene ricevuto entro 20 secondi, il sistema esegue l'attività dialISP in cui cerca di connettersi all' ISP. Appena tale attività termina, il sistema aspetta il carrier da parte dell' ISP. Se questo segnale non arriva entro 20 secondi, il sistema segnala la mancanza di carrier da parte dell' ISP e va in modo non connesso rimettendo il modem in modalità hook. Se il segnale arriva entro 20 secondi, il sistema viene connesso ed utilizza la connessione finché desidera.

Al termine della connessione il sistema viene fermato rimettendo il modem in modalità hook. Se in qualsiasi istante in cui cerca la connessione, il sistema riceve il segnale cancel, il sistema passa in stato non connesso.

Disegnare la macchina di stato UML che formalizza il comportamento del sistema ISPDialer.

Stati History

- Punto di ritorno a stati gerarchici precedentemente visitati

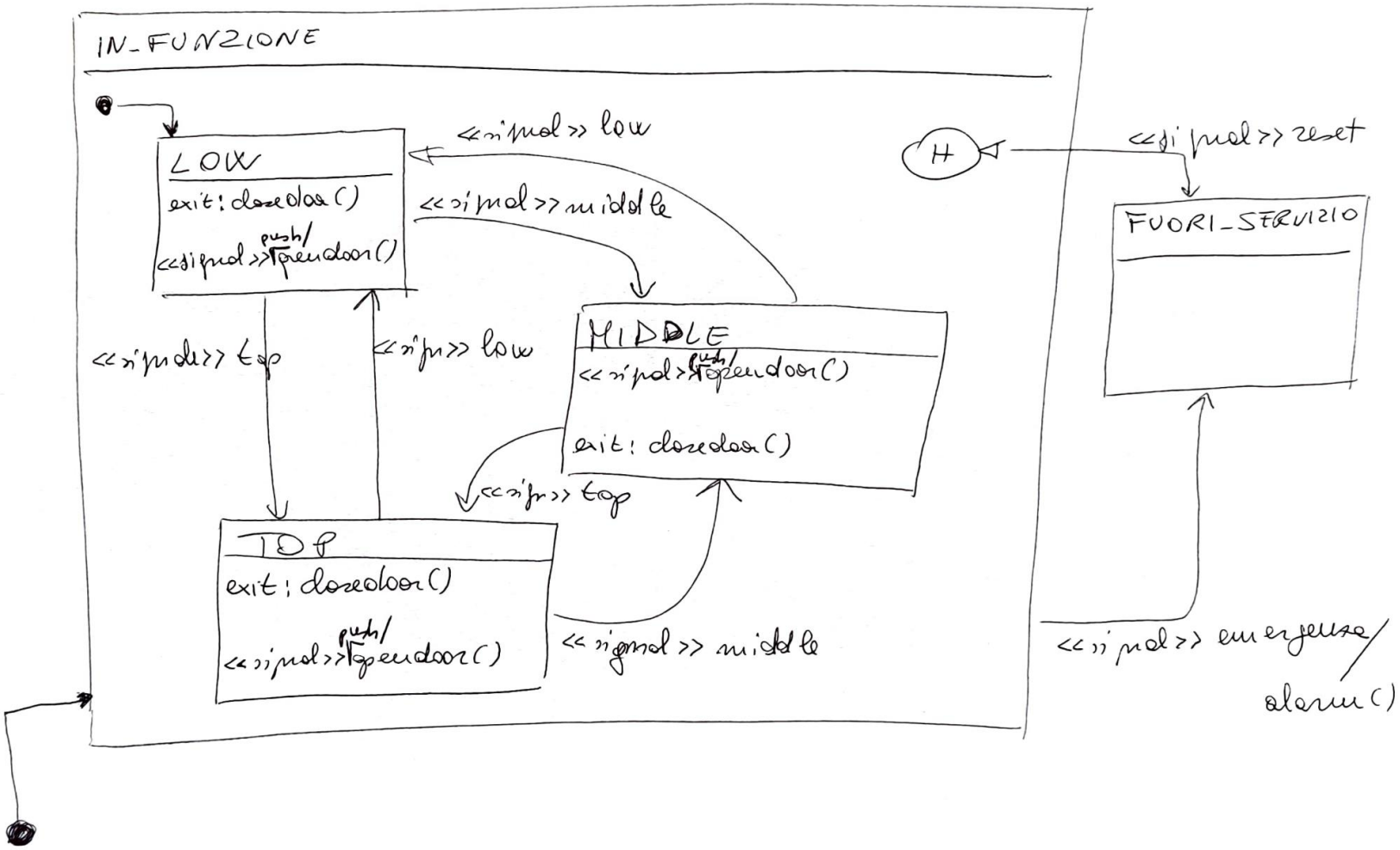


Esercizio

Utilizzando le macchine di stato UML modellare il comportamento di un *montacarichi* che si muove su tre piani -- top, middle, low -- in base alla richiesta che arriva da sensore. Per esempio, se il sensore segnala top, il montacarichi raggiungere la posizione più alta.

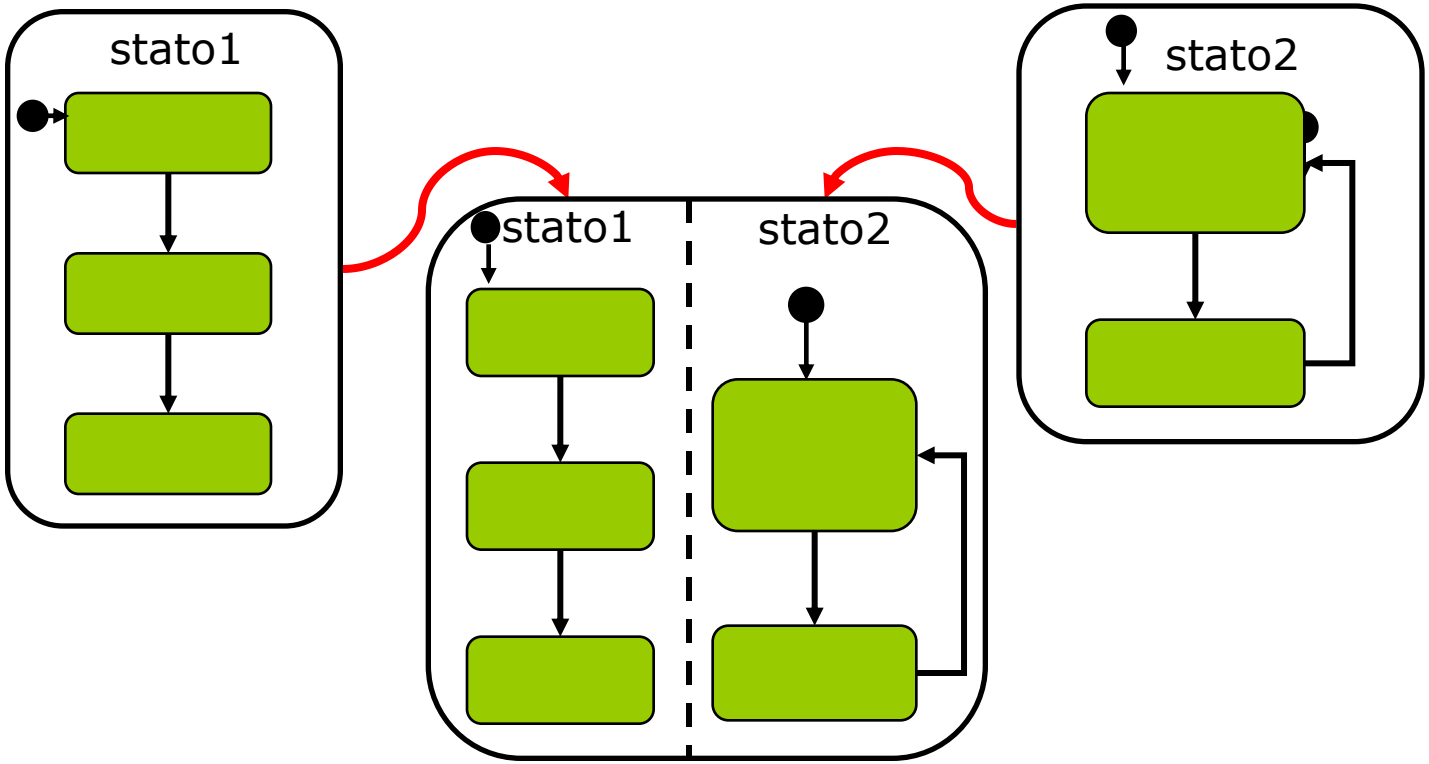
L'elevatore è dotato di un tasto per aprire le porte se il montacarichi è fermo, ed un tasto di emergenza che se pigiato causa il suono di una sirena e mette il montacarichi "fuori servizio". Quando è fuori servizio, se ripristinato, il montacarichi riparte dall'ultima configurazione lasciata.

Modello


IN_FUNZIONE
LOW
exit: closedoor()
<<signal>> push/ opendoor()
<<signal>> low
<<signal>> middle
H
<<signal>> reset
FUORI_SERVIZIO
<<signal>> top
<<signal>> low
MIDDLE
<<signal>> push/ opendoor()
exit: closedoor()
<<signal>> top
TOP
exit: closedoor()
<<signal>> push/ opendoor()
<<signal>> middle
<<signal>> emergenza/ alarm()

Stati composti paralleli

- Uno stato può contenere più regioni ortogonali
  - esse osservano uno stesso evento e reagiscono ad esso "simultaneamente"



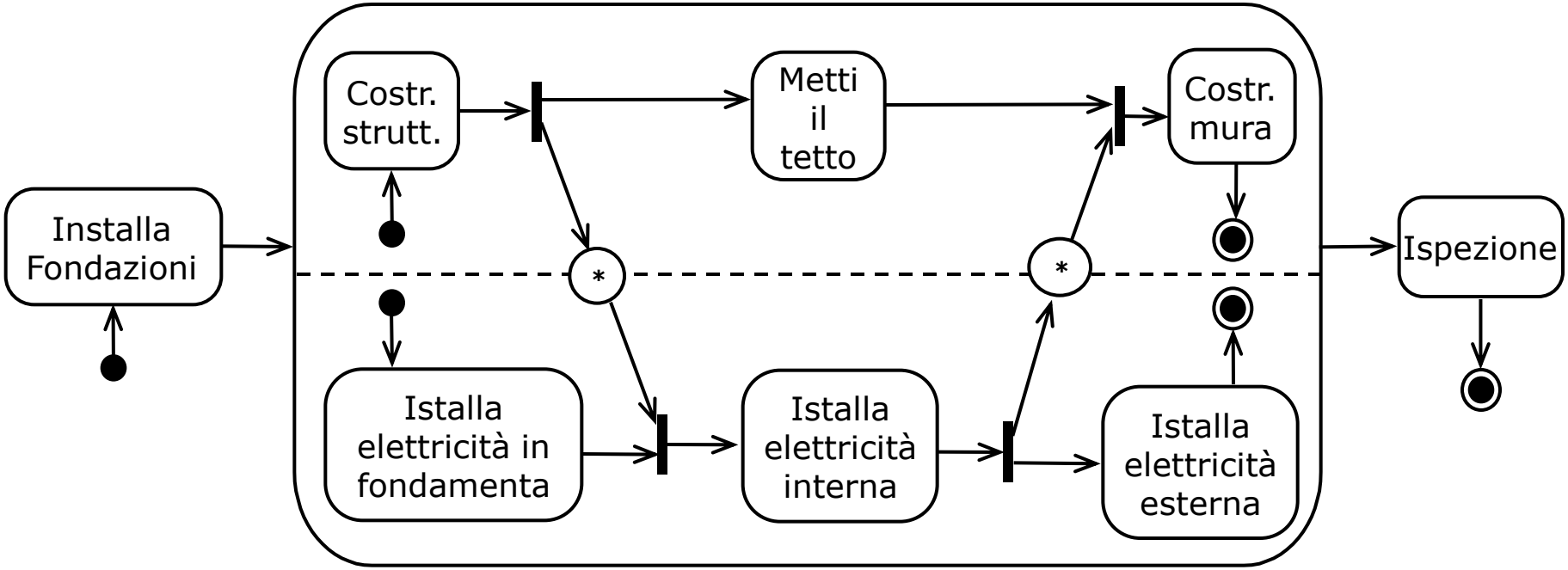
Interazione di diagrammi di stato (1)

La sincronizzazione tra diagrammi in regioni parallele avviene principalmente tramite il meccansmo di eventi/azioni:

- le **azioni** in un componente diventano **eventi** a cui altri componenti devono (possono) reagire interpretandoli come *call event*

Interazione di diagrammi di stato (3)

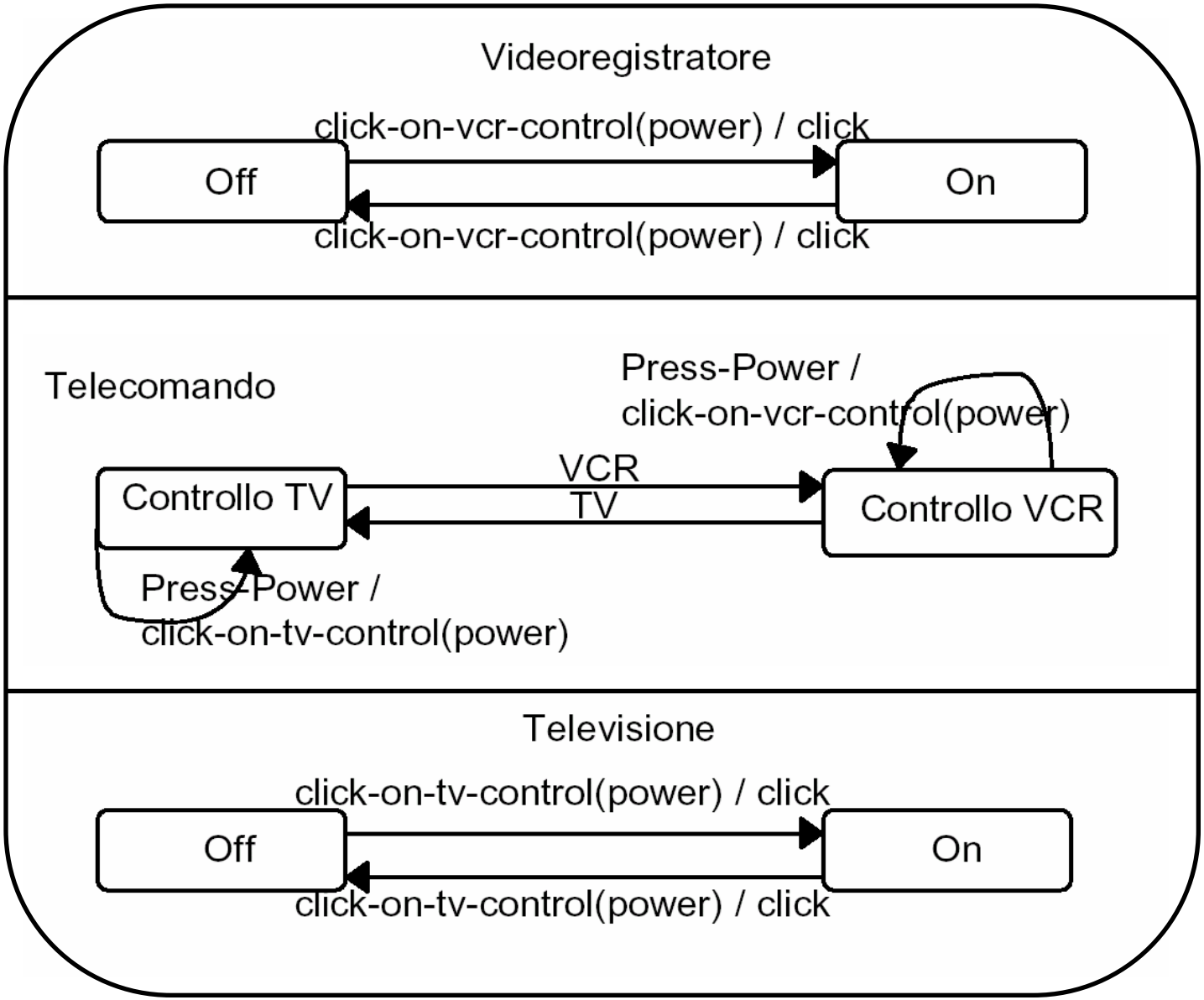
- Tramite stati di sincronizzazione:
  - synch state ⊛



Esempio: Interazione di diagrammi di stato (4)

- Vediamo una macchina di stato composta da un Telecomando che può controllare Televisione (in modalità ControlloTV) e Videoregistratore (in modalità ControlloVCR):
  - accendendo (evento Press-Power) in modalità ControlloTV, la televisione di accende
  - accendendo (evento Press-Power) in modalità ControlloVCR, il videoregistratore di accende

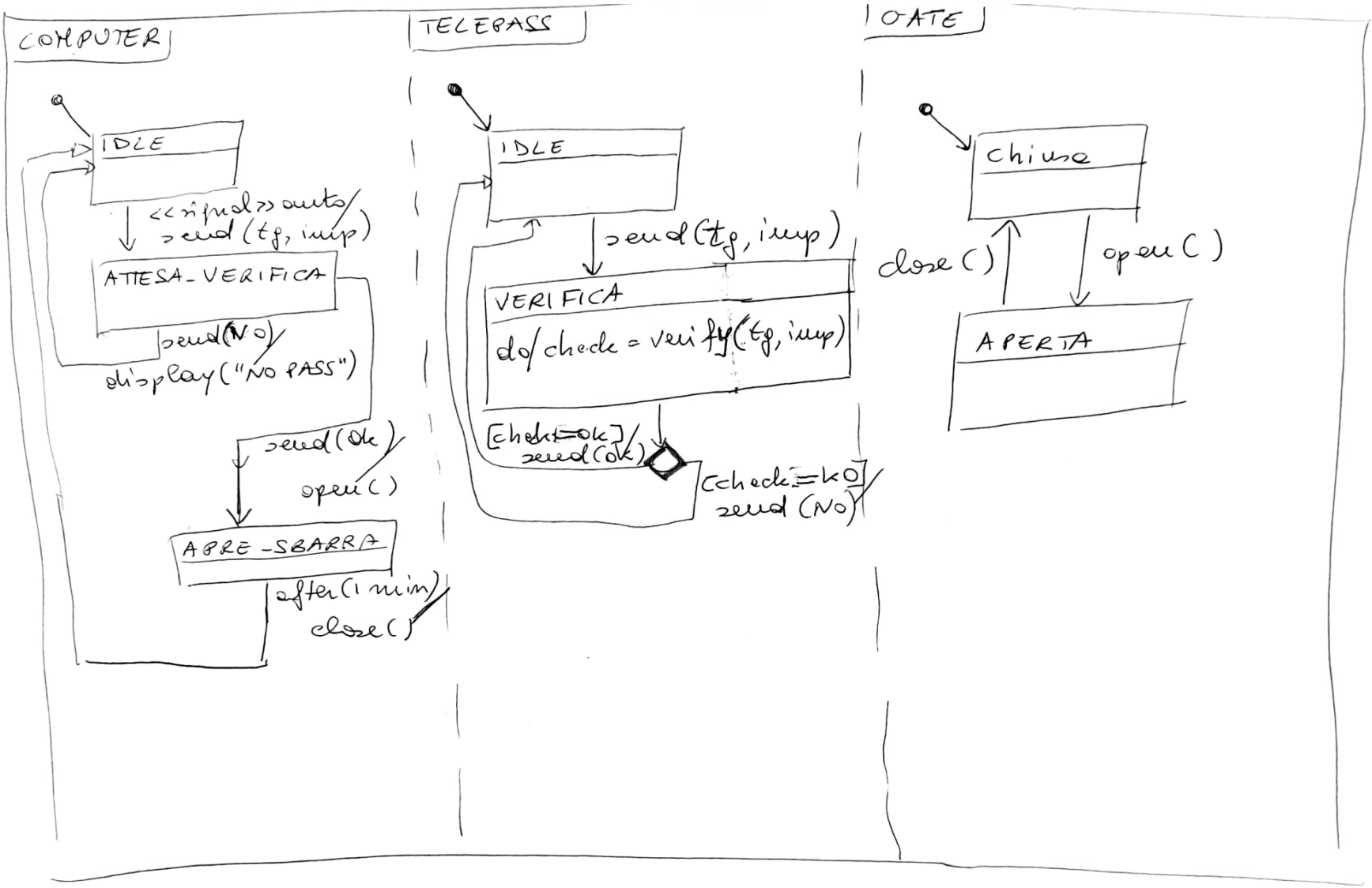
Interazione di diagrammi di stato (5)



Computer/telepass/sbarra

Utilizzando il formalismo delle macchine di stato UML, modellare il comportamento del seguente sistema di pagamento di pedaggio autostradale tramite telepass. All'arrivo di un'auto al casello autostradale, comunicata tramite sensore, un computer invia al sistema telepass l'informazione della targa della macchina e l'importo da pagare. Il sistema, fatti i dovuti controlli, invia al computer accettazione o rifiuto della richiesta. Ricevuta la risposta, il computer invia un segnale sul display di rifiuto del pagamento, oppure il comando di apertura alla sbarra. Questa provvede ad aprirsi ed a richiudersi dopo un minuto a seguito di opportuno comando del computer.

Modello


COMPUTER
TELEPASS
GATE
IDLE
<<signal>> auto/
send(tg, imp)
ATTESA_VERIFICA
send(NO)/
display("NO PASS")
send(ok)/
open()
APRE_SBARRA
after(1 min)
close()
IDLE
send(tg, imp)
VERIFICA
do/ check = verify(tg, imp)
[check=ok]/
send(ok)
[check=KO]/
send(NO)
Chiusa
close()
open()
APERTA

Montacarichi con controllore

Mediante una **state machine UML a componenti**, modellare il comportamento di un sistema composto da: **Controllore** che gestisce i comandi ricevuti dai pulsanti esterni e interni, inviando le istruzioni al montacarichi; **Elevatore** che può trovarsi in tre stati di posizione: **Low**, **Middle**, **Top**. Se uno dei pulsanti esterni (*Low*, *Middle*, *Top*) viene attivato, il controllore invia all'elevatore il comando di spostamento verso il piano richiesto. All'attivazione del pulsantieinterno per *Apertura delle porte*, il controllore invia opportuno comando all'elevatore. Se è premuto il *Tasto di emergenza*, su comando del controllore, il sistema va lin stato *Fuori Servizio*, e viene attivata la sirena di allarme.

Lavatrice

Tramite una macchina di stato UML, modellare il funzionamento di una lavatrice comandata tramite app. Il sistema è composto da una app e dalla lavatrice stessa, e può trovarsi in modalità di funzionamento *normale* ed in modalità di *manutenzione*. Durante il funzionamento normale, tramite comandi sul display dell'app, è possibile attivare il lavaggio, mettere in pausa il lavaggio, attivare o disattivare la centrifuga. Al termine del lavaggio, la lavatrice invia un segnale acustico di fine attività. In modalità manutenzione, vengono eseguiti in parallelo il controllo degli attuatori, e quello dei sensori. Terminato il controllo, il sistema ritorna in modalità di funzionamento normale.

Metropolitana

L'apertura e chiusura della porte di una metropolitana è controllata da un sistema costituito da due sensori (il sensore arrived che rileva l'arresto della metro alla stazione, ed il sensore pass_person che rileva il passaggio di una persona che vuole entrare nella metro quando le porte si stanno chiudendo) ed un controller. Su segnalazione del sensore arrived, il controller impiega da 1 a 3 tick di tempo per aprire le porte della metro inviando un apposito segnale alle porte e poi aspetta un segnale dalle porte di avvenuta apertura. La singola porta impiega da 2 a 4 tick per raggiungere la posizione open. Passato un minuto dall'apertura completa delle porte, il controller aziona la chiusura delle porte ed attende il segnale di avvenuta chiusura. La porta prende da 2 a 4 tick per chiudersi definitivamente. Se il sensore pass_person invia un segnale alla porta, la porta reagisce aprendosi completamente. In questo caso, il controller provvederà nuovamente alla chiusura delle porte.

Annaffiatoio automatico

Sfruttando il meccanismo di sincronizzazione tramite eventi, modellare con una macchina di stato UML il comportamento di un sistema d'irrigazione automatica. Il sistema è composto da un controllore e da un annaffiatoio. In modalità inverno il controllore invia il comando di erogazione dell'acqua ogni 12 ore e l'annaffiatoio eroga acqua per 15 minuti, dopo di che riceve il comando di interruzione di erogazione acqua. In modalità estate l'erogazione dell'acqua è richiesta ogni 8 ore per la durata di 30 minuti. Il controllore è dotato di un sensore di umidità. Se il segnale che arriva da tale sensore è alto, l'erogazione dell'acqua, per il ciclo corrente, è sospesa; se il segnale è basso, l'acqua è erogata, nel ciclo corrente, per ulteriori 5 minuti.

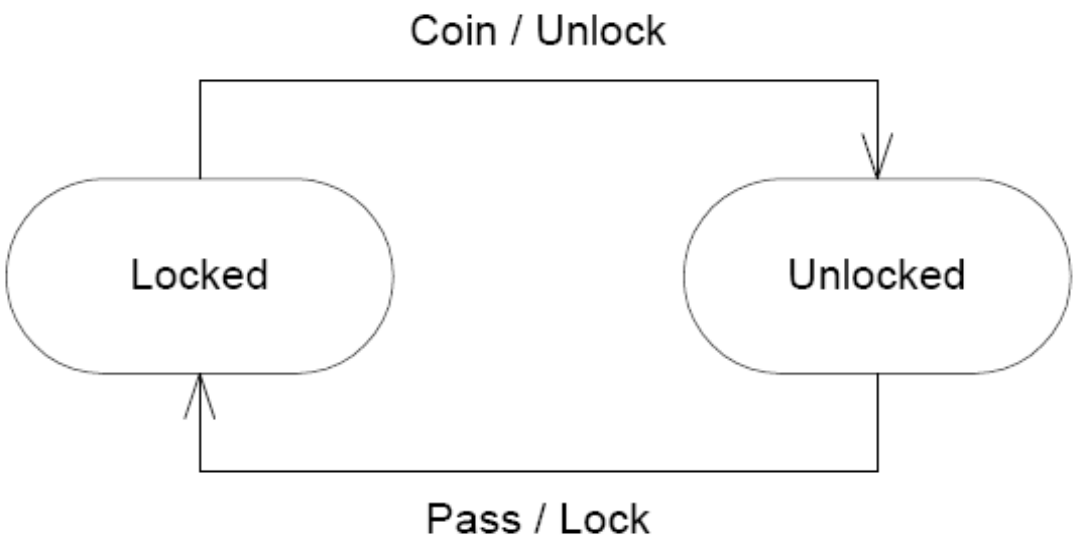
Esercizi

Vedi file **eserciziUMLStatechartsConcorrenti**

Caso di studio

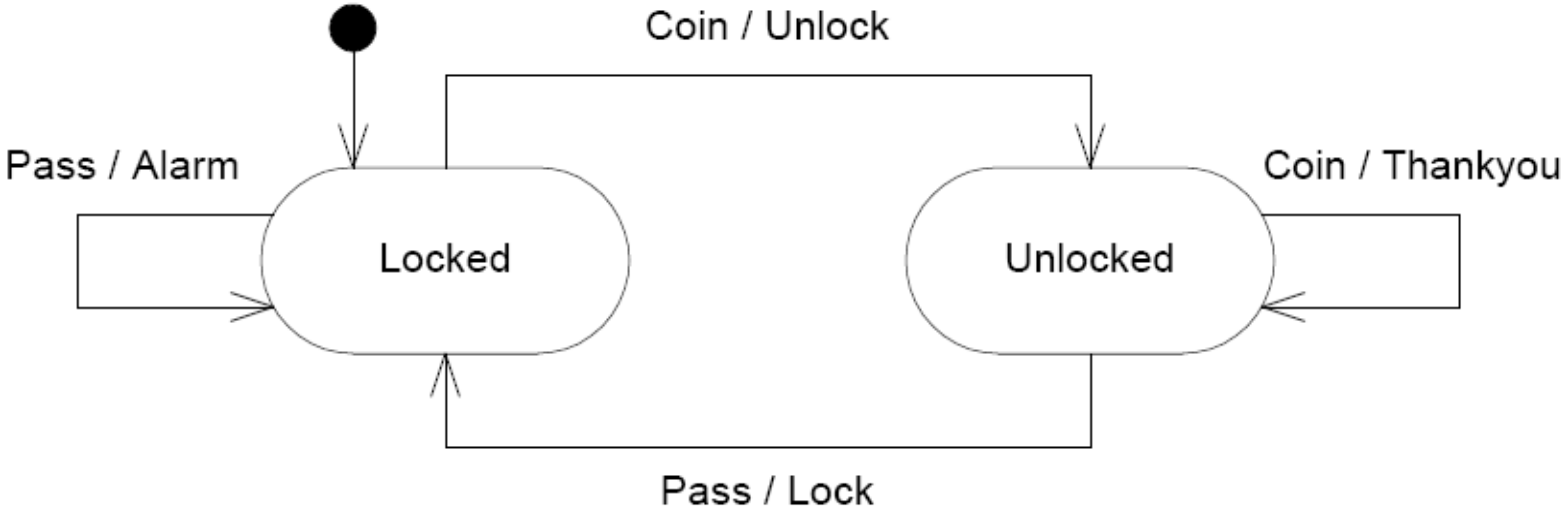
Tornello di una metro

- Il tornello può essere bloccato (locked) o non bloccato (unlocked)
- Quando è bloccato, la persona che deve passate inserisce una moneta nell'apposito slot ed il tornello si sblocca
- Quando la persona è passata, il tornello si blocca



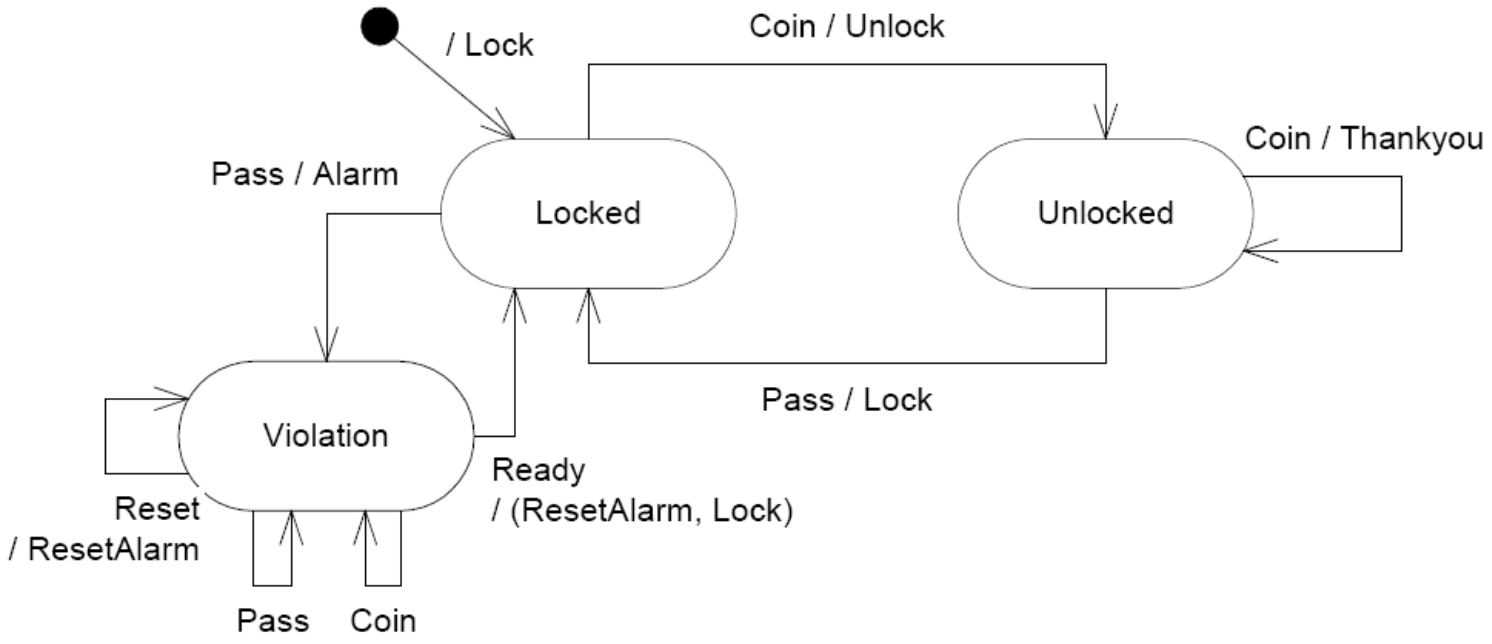
Comportamento anomali

- Una persona cerca di passare senza inserire la moneta: si attiva il sistema di allarme
- La persona inserisce una moneta quando già il tornello è sbloccato (ossia qualcuno ha già inserito una moneta)



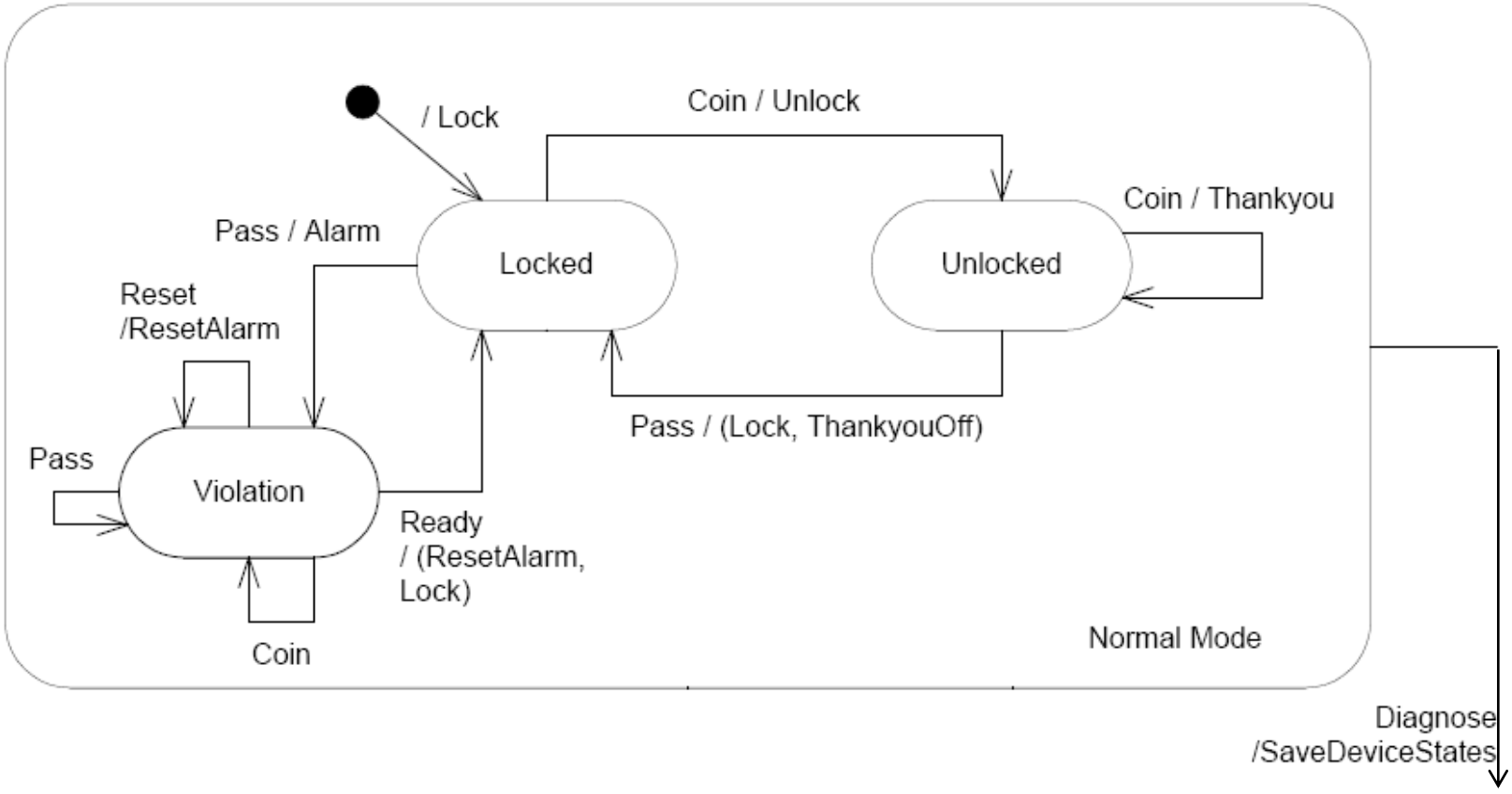
Gestione della situazione di allarme

- In caso di allarme, il sistema passa in una condizione di violazione e vi rimane finché non viene ripristinato il sistema e l'allarme viene spento



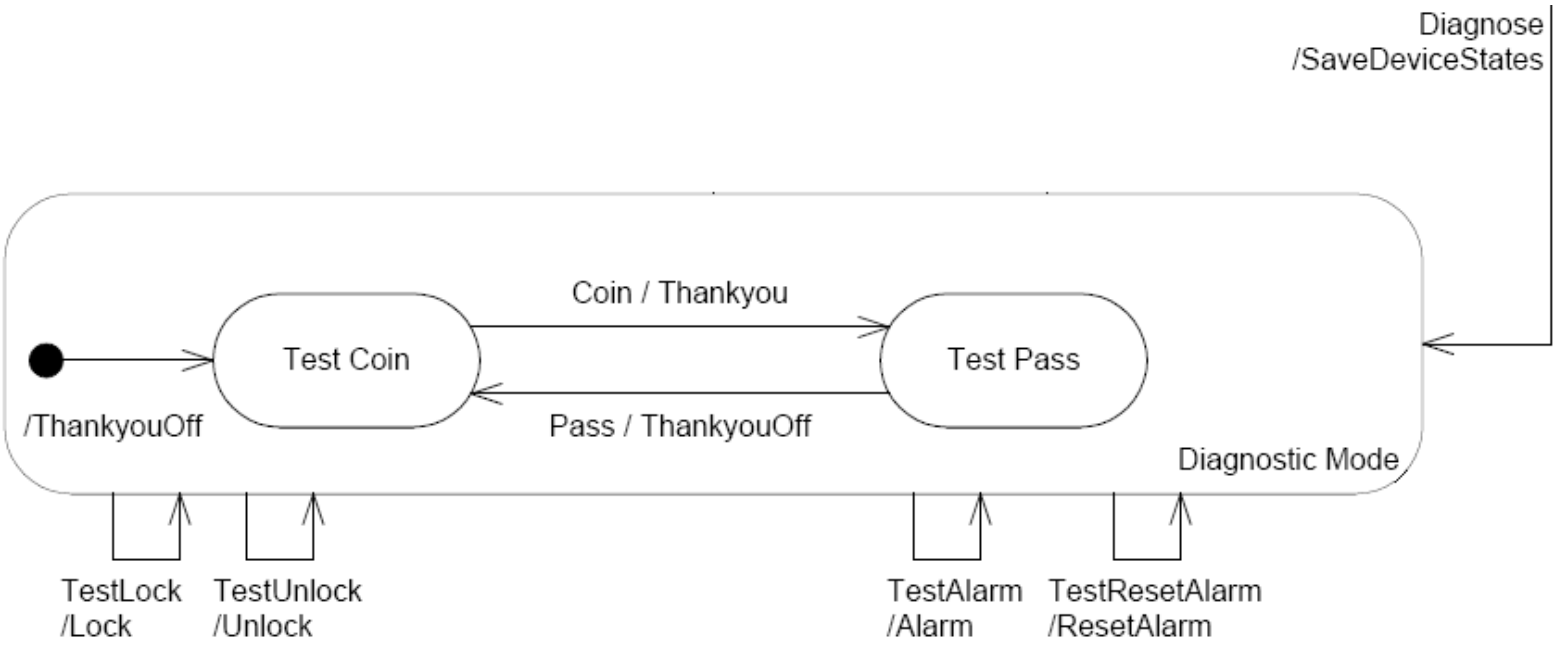
Manutenzione del tornello

- Quando occorre fare manutenzione, il sistema deve essere disattivato e passa in stato di diagnosi congelando lo stato del sistema

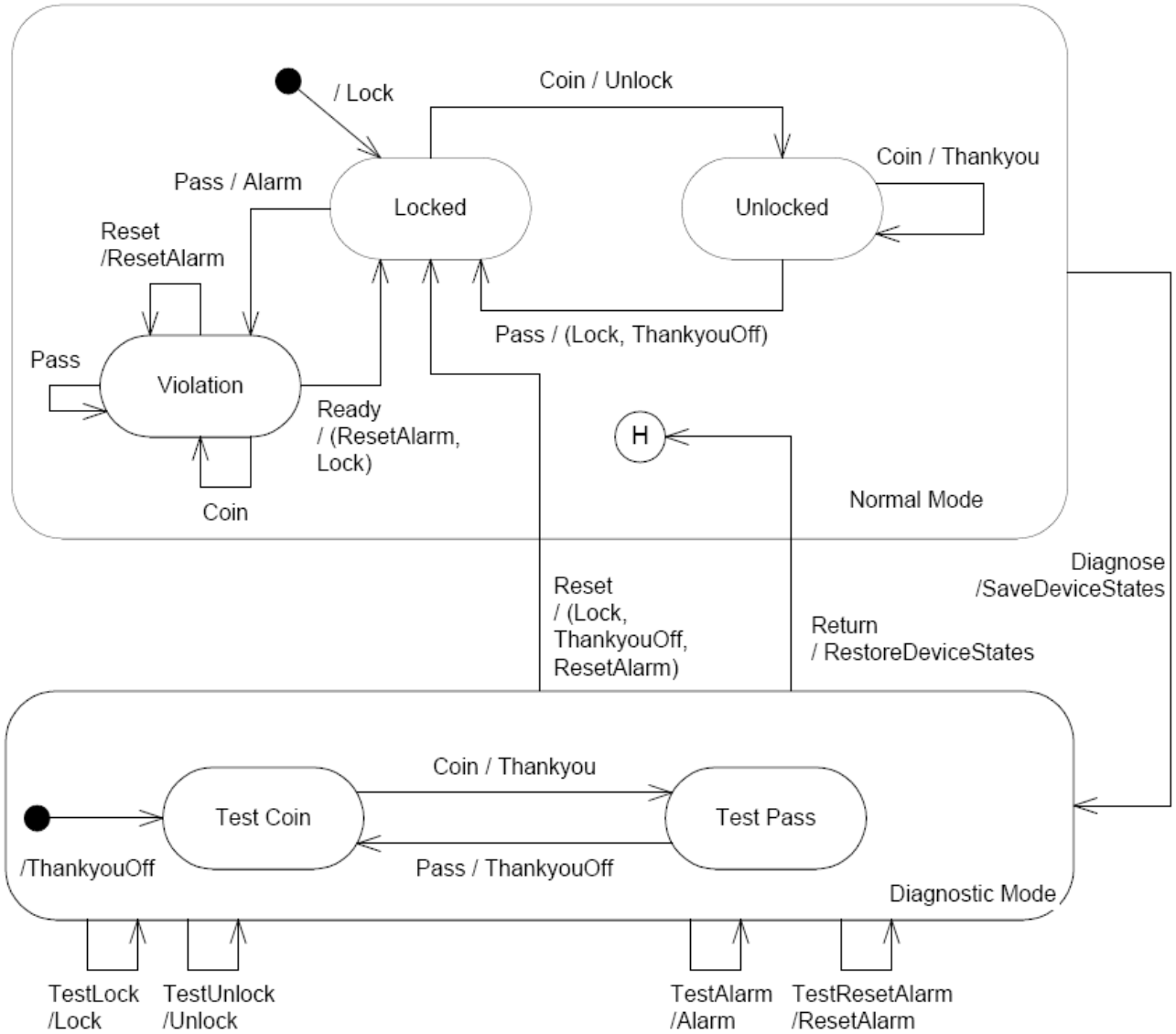


Diagnostica del tornello

- Durante la fase di diagnostica i tecnici controllano i sensori e tutte le funzionalità del dispositivo



Ripristino del sistema (1)

- Terminata la fase di diagnostica il dispositivo viene riattivato in due modi:
  - con tasto di Reset che pone il sistema in modo locked con allarme spento e luce di cortesia spenta
  - ripristinando la configurazione che il sistema aveva quando è passato in fase di diagnostica

Ripristino del sistema (2)